Swarm and Evolutionary Computation 78 (2023) 101283

Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo

APSM-jSO: A novel jSO variant with an adaptive parameter selection mechanism and a new external archive updating mechanism

Yintong Li [a], Tong Han [a], Huan Zhou [a,*], Yujie Wei [b,c], Yuan Wang [a], Mulai Tan [a], Changqiang Huang [a]

[a] Aviation Engineering school, Air Force Engineering University, Xi'an 710038, PR. China
[b] Air Defense and Antimissile School, Air Force Engineering University, Xi'an 710038, PR. China
[c] Air Force Xi'an Flying College, Xi'an 710300, PR. China

ARTICLE INFO

Keywords:
Evolutionary computation
Differential evolution
Global optimization
IEEE CEC 2018 test suite

ABSTRACT

A novel jSO variant named APSM-jSO is proposed in this study by making simple and effective modifications to improve its performance. There are three main differences between APSM-jSO and jSO. First, a novel adaptive selection mechanism (APSM) for selecting entries from the historical memory is designed to fully utilize the better entries in the historical memory. Subsequently, the first in-first out (FIFO) method is utilized to update the external archive for maintaining the population diversity and avoiding overuse of the external archive. Finally, a new mutation strategy adopting rank-based selective pressure (RSP) is used to enhance the exploitation of APSM-jSO. APSM-jSO is evaluated using the IEEE CEC 2018 test suite in comparison with five state-of-the-art DE-based variants (ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, mL-SHADE, and MadDE) and five winners of the IEEE CEC competitions (AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE). The results demonstrate that APSM-jSO outperforms jSO and five DE-based algorithms, is superior to four top winners of the IEEE CEC competitions (AGSK, APGSK-IMODE, HSES, and IMODE), and is not inferior to the top method of the IEEE CEC 2017 competition (EBOwithCMAR). The conclusion is that APSM-jSO effectively further enhances the overall performance of jSO, and is an excellent jSO variant. The MATLAB source code of APSM-jSO can bedownloaded from https://github.com/Yintong-Li/APSM-jSO.

1. Introduction

In recent decades, evolutionary algorithms (EAs) have been shown to be more effective than traditional search algorithms based on gradient information in generating optimal or near-optimal solutions for rough optimization [1,2]. Moreover, EAs have been widely used to solve optimization problems in science and engineering because they are global, robust, and can be easily applied to any problem [3–8]. Differential Evolution (DE), one of the most widely used EAs in the fields of engineering, statistics, and finance, is a population-based evolutionary algorithm proposed by Price [9] in 1995. However, DE has certain drawbacks that limit its performance and applications. The main drawback of DE includes sensitivity to parameter settings and easy falling into local optima.

To overcome the above-mentioned drawbacks of DE and its variants, a series of improvements have been proposed in various ways over the past decades. Researchers have provided a variety of competitive DE variants whose quality can be easily tracked through the results of the IEEE conference on evolutionary computing (CEC), where DE-based algorithms are ranked steadily at the top [10]. According to the methods of improvement, the most effective improvements are divided into two categories [11]: adaptive parameter settings and modification for the mutation strategy.

(a) Adaptive parameter settings. In [12], a self-adapting control-parameter strategy was introduced into DE by Brest et al., in 2006 to propose jDE for adjusting the most important parameters including scaling factor and crossover rate. Tanabe and Fukunaga [13] proposed a success-history-based parameter adaptation for DE (SHADE) in 2013 using an external archive of inferior solutions from JADE [14] and success-historical memory to adjust scaling factor F and crossover rate Cr. Since 2013, SHADE has been considered as the denominator for developing promising DE variants. Tanabe and Fukunaga [15] proposed LSHADE based on SHADE in 2014, by introducing a linear population size reduction strategy (LPSR) to balance exploration and exploitation.

* Corresponding author.
E-mail address: kgy_zhouh@163.com (H. Zhou).

https://doi.org/10.1016/j.swevo.2023.101283
Received 8 September 2022; Received in revised form 21 December 2022; Accepted 16 February 2023
Available online 25 February 2023
2210-6502/© 2023 Elsevier B.V. All rights reserved.

Brest et al. [16] proposed iL-SHADE in 2016 by modifying its memory update mechanism, in which higher values for *CR* and lower values for *F* were propagated to enrich the population diversity at the early stages of evolution. In 2017, Brest et al. [17] proposed jSO based on iL-SHADE by introducing a new parameter F_w based on the scaling factor *F* to the mutation strategy for tuning the exploitation ability and the exploration ability. In [18], LSHADE-EpSin was proposed by adopting the integrated sine method to automatically adjust the value of the scaling factor to balance the exploration ability and the exploitation ability, and utilizing the Gaussian walk-based local search method to enhance the exploitation ability at later generations. Moreover, they proposed LSHADE-cnEpSin [19] to further improve LSHADE-EpSin by adopting an adaptation scheme based on earlier success for the ensemble of sinusoidal approaches, and covariance matrix learning with Euclidean neighborhood for the crossover operator was used to enhance its capability for dealing with non-separable problems. In [20], LSHADE-RSP was proposed based on jSO by adopting a rank-based selective pressure strategy (RSP) instead of the randomly selected individuals used in the mutation strategy by Stanovov et al.

(a) Modification of mutation strategy. Zhang and Sanderson [14] proposed JADE in 2009 by adopting a new mutation strategy "DE/current-to-*p*best" with an optional external archive of inferior solutions and updating control parameters in an adaptive manner. Wang et al., proposed the CoDE [21] in 2011 by randomly combining three mutation strategies with different parameter settings for generating trial vectors for different problems. Zhao et al., proposed the LSHADE-CLM [22] in 2020 by using a cooperative mutation mechanism between "*DE/current−to−pbetter/r*" with comprehensive learning mechanism and "*DE/current−to−pbest−w/1*", and the search direction was guided by utilizing the population covariance matrix. Choi and Ahn [3] proposed iLSHADE-RSP by adopting the Cauchy distribution based on a jumping rate to perturb the target vector and generate a trial vector with great diversity in the mutation strategy. In order to better guide the research of differential evolution mutations, Mohamed, et al. [23] made a comprehensive analysis of the contributions on mutation strategies proposed between 1995 and 2020. Cui, et al. [24] proposed a composite differential evolution, named AMECoDEs, by employing two elites-guided mutation strategies to guide better evolutionary direction and a shift mechanism to handle falling into local optima. Mohamed, et al. [25] introduced a new triangular mutation operator, that is based on the difference vectors between the best, better, and worst individuals among three randomly selected vectors, to propose a novel DE variant named EFADE to better balance the exploration and exploitation. Mohamed, et al. [26] proposed a new mutation rule for DE, in which the population were divided into three subpopulations to support the mutation process, to balance the exploration and exploitation of DE. In [27], a multi-population-based framework (MPF) was proposed to realize the high-level ensemble of three excellent DE variants including JADE [14], CoDE [21], and EPSDE [28]. In [29], a variant of DE named FDDE was proposed by modifying the mutation strategy, in which the fitness and their diversity contribution of individuals are considering to assign suitable position for each individual during the mutation strategy. In order to further balance the exploitation and exploration, an adaptive differential evolution named EJADE [30] was proposed by combining two sets of mutation and crossover strategies, and introducing an adaptive parameter control strategy to enhance the global search ability. In [31], an improved framework for variant of SHADE was proposed by making a modification to scaling factor and crossover rate adaptation, in which a distance-based parameter adaptation is designed to alleviate the premature convergence and enhance the exploration. In addition, some studies have also hybridized DE and its variants with other EAs [32–34].

The above-mentioned variants of DE improve its performance in different ways. In particular, the jSO algorithm significantly improves its performance in the basic DE framework. In this paper, APSM-jSO, a novel jSO variant, is proposed for further improvement. The main contributions can be summarized as follows. (1) A novel adaptive selection mechanism (APSM) for selecting entries from the historical memory is designed to fully utilize the better entries in the historical memory. (2) The first in-first out (FIFO) method is utilized to update the external archive for maintaining the population diversity and avoiding the overuse of external archive. (3) A new RSP-based mutation strategy is used to strengthen the exploitation of APSM-jSO.

The remainder of this work is structured as follows. DE and its state-of-the-art variants are described in Section 2. Section 3 describes the details of APSM-jSO proposed in this work. The numerical experiments and comparisons based on the IEEE CEC 2018 test suite are implemented in Section 4. Finally, Section 6 makes conclusions and future work for this paper.

2. DE and its state-of-the-art variants

Because APSM-jSO is proposed based on the state-of-the-art variant of DE, we firstly make a description of the canonical DE proposed by Storn and Price [9] in this section. The most competitive DE-based variants are shown in Table 1.

The development path from DE to the most competing DE variants, SHADE, L-SHADE, iL-SHADE, jSO, and LSHADE-RSPare briefly described as follows.

2.1. Differential evolution

Canonical DE [9] is a population-based evolutionary algorithm in which three operators, mutation, crossover, and selection operations, are executed during the evolutionary process.

The initial population $\boldsymbol{P}$ is generated using a uniform distribution as shown in Eq. (1).

$$x_i^j = L^j + rand\cdot\left(U^j - L^j\right) \tag{1}$$

where $rand \in (0,1)$ is a uniformly distributed random number. $i \in \{1, 2, 3, \cdots, NP\}$ is the individual index and *NP* is the population size, $j \in$

Table 1
Most competitive DE-based variants.

Algorithm	Years	Description
L-SHADE	2014	This algorithm is a modification of SHADE by adopting LPSR to adjust the population size during the evolution [15].
iL-SHADE	2016	This algorithm enhances L-SHADE by adopting the weighted Lehmer means to update historical memory, tuning *Cr* and *F* value for different stage of evolutionary process, and using linear increase greediness factor *p* [16].
jSO	2017	This algorithm enhances iL-SHADE by adopting a new mutation strategy "*DE/current-to-pbest-w/1*" [17].
LSHADE-RSP	2018	This algorithm is a modification of jSO by adopting a RSP strategy and making some changes to the scaling factor and crossover rate [20].
mL-SHADE	2019	This algorithm is a modification of L-SHADE by removing the terminal value, and adding a polynomial mutation and a memory perturbation mechanism [35].
EB-LSHADE	2019	This algorithm improves L-SHADE by adopting a novel mutation strategy DE/current-to-ord_pbest/ [36].
LSHADE-CLM	2020	This algorithm is proposed by using a cooperative mutation mechanism between "*DE/current−to−pbetter/r*" and "*DE/current−to−pbest−w/1*", and the search direction was guided by utilizing the population covariance matrix [22].
ELSHADE-SPACMA [37]	2021	This algorithm is an enhanced version of LSHADE-SPACMA [32] by adopting a dynamic *p*-value and a directed mutation strategy. Specifically, it is composed of the combination of LSHADE-SPACMA [32] and AGDE [26].
MadDE	2021	This algorithm is a variant of LSHADE by combining multiple mutation strategies to leverage the power of multiple adaptation strategies [38].

$\{1, 2, 3, \cdots, D\}$ is the dimension index and D is the dimensionality, L^j and U^j are problem-specific lower and upper boundaries on j^{th} dimension, respectively [11].

Four widely used classical mutation strategies, *DE/best/2*, *DE/rand/2*, and *DE/current-to-best/1*, *DE/current-to-rand/1*, in DE for generating the mutant vector $\boldsymbol{v_i}$ are described as Eqs. (2)–(5).

$$\boldsymbol{v}_i = \boldsymbol{x}_{best} + F\cdot(\boldsymbol{x}_{r_1} - \boldsymbol{x}_{r_2}) + F\cdot(\boldsymbol{x}_{r_3} - \boldsymbol{x}_{r_4}) \tag{2}$$

$$\boldsymbol{v}_i = \boldsymbol{x}_{r_1} + F\cdot(\boldsymbol{x}_{r_2} - \boldsymbol{x}_{r_3}) + F\cdot(\boldsymbol{x}_{r_4} - \boldsymbol{x}_{r_5}) \tag{3}$$

$$\boldsymbol{v}_i = \boldsymbol{x}_i + F\cdot(\boldsymbol{x}_{best} - \boldsymbol{x}_i) + F\cdot(\boldsymbol{x}_{r_1} - \boldsymbol{x}_{r_2}) \tag{4}$$

$$\boldsymbol{v}_i = \boldsymbol{x}_i + K\cdot(\boldsymbol{x}_{r_1} - \boldsymbol{x}_i) + F\cdot(\boldsymbol{x}_{r_3} - \boldsymbol{x}_{r_4}) \tag{5}$$

where $r_1, r_2, r_3, r_4, r_5 \in \{1, 2, 3, \cdots, NP\}$ are the uniformly distributed random indices, which differ from each other. F is the scaling factor, and $\boldsymbol{x}_{best}$ is the best individual obtained so far. Furthermore, a correction strategy is adopted as shown in Eq. (6) once $\boldsymbol{v_i}$ is out of the lower or upper boundaries.

$$v_i^j = \begin{cases} (L^j + x_i^j)/2, & v_i^j < L_j \\ (U^j + x_i^j)/2, & v_i^j > U_j \end{cases} \tag{6}$$

The binomial crossover operation is adopted to generate the trial vector $\boldsymbol{u_i}$ based on the mutant vector, as shown in Eq.(7).

$$u_i^j = \begin{cases} v_i^j, & rand < Cr \text{ or } j = j_{rand} \\ x_i^j, & \text{otherwise} \end{cases} \tag{7}$$

where $j_{rand} \in \{1, 2, 3, \cdots, D\}$ denotes a uniformly random index and Cr is the crossover rate.

The selection operator based on the greedy strategy is utilized to update the population, as shown in Eq.(8).

$$\boldsymbol{x}_i = \begin{cases} \boldsymbol{u}_i, & f(\boldsymbol{u}_i) \le f(\boldsymbol{x}_i) \\ \boldsymbol{x}_i, & \text{otherwise} \end{cases} \tag{8}$$

where $f(\cdot)$ denotes the fitness function of a specific problem.

2.2. SHADE

SHADE [13] was proposed in 2013 by adopting an external archive scheme and history-based parameter adaptation scheme. The external archive $\boldsymbol{A}$ was introduced into DE to enrich the population diversity. The target vector $\boldsymbol{x}_i$ will be saved in the external archive if the trial vector $\boldsymbol{u}_i$ is better than it. Otherwise, it will be preserved in the next generation, as shown in Eq.(8). Once size $|\boldsymbol{A}|$ of the archive reaches a predefined size, a random individual will be discarded to make room for the newly inserted individual.

The history-based parameter adaptation scheme is designed for adjusting F_i and Cr_i. All successful F_i and Cr_i are saved into $\boldsymbol{S}_{\text{CR}}$ and $\boldsymbol{S}_F$ to update the kth pair of elements in the historical memory $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_F$ by using the weighted Lehmer mean in each generation, as shown in Eqs. (9)–(12). k starts at one and is incremented by one at each generation. Moreover, k is reset to 1 if it overflows the memory size H [13]. In one generation, $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_F$ are not updated if all trial vectors are worse than the corresponding target vectors.

$$M_{F,k} = \begin{cases} M_{F,k}, & S_F = \varnothing \\ \text{mean}_{WL}(S_F), & \text{otherwise} \end{cases} \tag{9}$$

$$M_{CR,k} = \begin{cases} M_{CR,k}, & S_{CR} = \varnothing \\ \text{mean}_{WL}(S_{CR}), & \text{otherwise} \end{cases} \tag{10}$$

$$\text{mean}_{WL}(\boldsymbol{S}) = \frac{\sum_{n=1}^{|S|} \omega_n \cdot S_n^2}{\sum_{n=1}^{|S|} \omega_n \cdot S_n} \tag{11}$$

$$\omega_n = \frac{|f(\boldsymbol{u}_n) - f(\boldsymbol{x}_n)|}{\sum_{n=1}^{|S|} |f(\boldsymbol{u}_n) - f(\boldsymbol{x}_n)|} \tag{12}$$

In Eq.(11), $\boldsymbol{S}$ stands for $\boldsymbol{S}_{CR}$ or $\boldsymbol{S}_F$. F_i and Cr_i for individual $\boldsymbol{x}_i$ are created by sampling from the Cauchy distribution as shown in Eq.(13) and the normal distribution as shown in Eq.(15), respectively. F_i and Cr_i are further corrected using Eq.(14) and Eq.(16), respectively.

$$F_i = randc(M_{F,R_i}, 0.1) \tag{13}$$

$$F_i = \begin{cases} 1, & F_i > 1 \\ \text{Applying Eq.(13)}, & F_i \le 0 \end{cases} \tag{14}$$

$$Cr_i = randn(M_{CR,R_i}, 0.1) \tag{15}$$

$$Cr_i = \begin{cases} 1, Cr_i > 1 \\ 0, Cr_i < 0 \end{cases} \tag{16}$$

where $R_i \in \{1, 2, 3, \cdots H\}$ denotes the random index. Moreover, the mutation strategy "*DE/current-to-pbest/1*" with the external archive is used for generating the mutant vector $\boldsymbol{v_i}$ in SHADE, as shown in Eq. (17).

$$\boldsymbol{v}_i = \boldsymbol{x}_i + F_i\cdot(\boldsymbol{x}_{pbest} - \boldsymbol{x}_i) + F_i\cdot\left(\boldsymbol{x}_{r_1} - \tilde{\boldsymbol{x}}_{r_6}\right) \tag{17}$$

where $\boldsymbol{x}_{pbest}$ denotes a random individual selected from the dominant population including top $NP\times p$ individuals, and p denotes the greediness factor. $\tilde{\boldsymbol{x}}_{r_6}$ is a random individual selected from the combination of $\boldsymbol{A}$ and $\boldsymbol{P}$, which differs from $\boldsymbol{x}_i$, $\boldsymbol{x}_{pbest}$, or $\boldsymbol{x}_{r1}$.

Because the crossover rate is different for individuals, Eq.(7) has been modified as Eq.(18) in SHADE.

$$u_i^j = \begin{cases} v_i^j, & rand < Cr_i \text{ or } j = j_{rand} \\ x_i^j, & \text{otherwise} \end{cases} \tag{18}$$

where Cr_i is the crossover rate for i^{th} mutant vector $\boldsymbol{v_i}$. Moreover, each individual $\boldsymbol{x}_i$ has an associated p_i that is computed using Eq.(19) by generation.

$$p_i = \max\{2/NP, 0.2\cdot rand\} \tag{19}$$

2.3. L-SHADE

L-SHADE [15] was proposed by introducing a linear population size reduction (LPSR) strategy to SHADE to balance exploitation and exploration. In L-SHADE, the population size NP is updated using a linear function during the evolution, as shown in Eq. (20).

$$NP = \text{round}\left[\frac{FEs}{FEs_{max}}(NP_{\min} - NP_{\text{init}}) + NP_{\text{init}}\right] \tag{20}$$

where NP_{min} and NP_{init} are the minimum and maximum population sizes. $FEs_{\max}$ and FEs are the maximum and current number of evaluations, respectively. The worst individuals are discarded from $\boldsymbol{P}$ at the end of each generation to meet the updating NP.

In addition, there are some modifications to the historical memory $\boldsymbol{M}_{CR}$ update as shown in Eq.(21). Specifically, as $\boldsymbol{M}_{CR}$ is updated, if $\boldsymbol{M}_{CR,k}$ is equal to the terminal value $\perp$ or max($\boldsymbol{S}_{CR}$)=0), then $\boldsymbol{M}_{CR,k}$ is set to $\perp$. Thus, if $\boldsymbol{M}_{CR}$ is assigned $\perp$, $\boldsymbol{M}_{CR}$ will fixed at terminal value that will lock Cr_i to 0 until the end of the search, as shown in Eq.(22).

$$M_{CR,k} = \begin{cases} \perp, & M_{CR,k} = \perp \ or \ \max(\boldsymbol{S}_{CR} = 0) \\ \text{mean}_{WL}(\boldsymbol{S}_{CR}), & \text{otherwise} \end{cases} \tag{21}$$

$$Cr_i = \begin{cases} 0, & M_{CR,R_i} = \perp \\ \text{randn}\left(M_{CR,R_i}, 0.1\right), & \text{otherwise} \end{cases} \tag{22}$$

In L-SHADE, the greediness factor p is set to a constant instead of being generated using Eq.(19). The external archive size is set to $NP \times A_{\text{rate}}$ and A_{rate} is a control parameter for the external archive size. Note that the external archive size is updated based on NP during the evolution.

2.4. iL-SHADE

iL-SHADE [16] is an extended version of L-SHADE. The main features that differ from those of LSHADE are as follows.

Firstly, the $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_F$ of the current generation are considered to update the historical memory as shown in Eq.(23) and Eq.(24). All entries in $\boldsymbol{M}_{CR}$ are set to 0.8 at the beginning (they are set to 0.5 in L-SHADE) and one entry of the historical memory maintains fixed values, that is, $\boldsymbol{M}_{CR,H}$ and $\boldsymbol{M}_{F,H}$ are always set to 0.9 during the evolutionary process.

$$M_{F,k} = \begin{cases} M_{F,k}, & \boldsymbol{S}_F = \varnothing \\ \dfrac{\left(\text{mean}_{WL}(\boldsymbol{S}_F) + M_{F,k}\right)}{2}, & \text{otherwise} \end{cases} \tag{23}$$

$$M_{CR,k} = \begin{cases} \perp, & M_{CR,k} = \perp \ \ or \ \ \max(\boldsymbol{S}_{CR} = 0) \\ \dfrac{\left(\text{mean}_{WL}(\boldsymbol{S}_{CR}) + M_{CR,k}\right)}{2}, & \text{otherwise} \end{cases} \tag{24}$$

Second, the very low values of Cr_i and the very high values of F_i are limited in the early stages of the evolution [16]. Specifically, Cr_i and F_i generated using Eq.(22) and Eq.(13) will be adjusted as follows.

$$Cr_i = \begin{cases} \max(Cr_i, 0.5), & FEs < 0.25 \cdot FEs_{\max} \\ \max(Cr_i, 0.25), & 0.25 \cdot FEs_{\max} \le FEs < 0.5 \cdot FEs_{\max} \end{cases} \tag{25}$$

$$F_i = \begin{cases} \min(F_i, 0.7), & FEs < 0.25 \cdot FEs_{\max} \\ \min(F_i, 0.8), & 0.25 \cdot FEs_{\max} \le FEs < 0.5 \cdot FEs_{\max} \\ \min(F_i, 0.9), & 0.5 \cdot FEs_{\max} \le FEs < 0.75 \cdot FEs_{\max} \end{cases} \tag{26}$$

Finally, the greediness factor p for all individuals in one generation is generated using Eq.(27).

$$p = p_{\min} + (p_{\max} - p_{\min}) \cdot \frac{FEs}{FEs_{\max}} \tag{27}$$

In iL-SHADE, $p_{\max}$=0.2 and $p_{\min}$=0.1 during the evolutionary process.

2.5. jSO

jSO [17] is an improved variant of iL-SHADE that adopts a new weighted version of mutation strategy "*DE/current-to-pbest-w/1*", which is described as shown in Eq.(28).

$$\boldsymbol{v}_i = \boldsymbol{x}_i + Fw_i \cdot \left(\boldsymbol{x}_{pbest} - \boldsymbol{x}_i\right) + F_i \cdot \left(\boldsymbol{x}_{r_1} - \boldsymbol{x}_{r_2}\right) \tag{28}$$

$$Fw_i = \begin{cases} 0.7 \cdot F_i, & FEs < 0.2 \cdot FEs_{\max} \\ 0.8 \cdot F_i, & 0.2 \cdot FEs_{\max} \le FEs < 0.4 \cdot FEs_{\max} \\ 1.2 \cdot F_i, & FEs \ge 0.4 \cdot FEs_{\max} \end{cases} \tag{29}$$

In the new mutation strategy, a smaller factor Fw_i is adopted at the early stages of the evolutionary process, whereas in later stages, a higher factor Fw_i is used to tune exploration and exploitation [20].

Moreover, $p_{\max}$=0.25 and $p_{\min}$=$p_{\max}$/2 during the evolutionary process, and the $\boldsymbol{M}_F$ values are initialized to 0.3 in jSO.

2.6. LSHADE-RSP

LSHADE-RSP is a recently enhanced version of jSO by introducing RSP and adjusting F_i and Cr_i. The RSP-based mutation strategy, *DE/current-to-pbest-w/r*, is described as Eq.(30).

$$\boldsymbol{v}_i = \boldsymbol{x}_i + Fw_i \cdot \left(\boldsymbol{x}_{pbest} - \boldsymbol{x}_i\right) + F_i \cdot \left(\boldsymbol{x}_{pr_1} - \boldsymbol{x}_{pr_2}\right) \tag{30}$$

where $\boldsymbol{x}_{pr1}$ denotes a vector selected from the population $\boldsymbol{P}$ based on rank-based probabilities, $\boldsymbol{x}_{pr2}$ is a vector selected from $\boldsymbol{P}$ based on rank-based probabilities or randomly selected from $\boldsymbol{A}$. The selected probability of individual i is calculated using Eq.(31).

$$Pr_i = Rank_i \,/\, (Rank_1 + Rank_2 + \cdots + Rank_{NP}) \tag{31}$$

$$Rank_i = k(NP - i) + 1 \tag{32}$$

where the largest and smallest rank are assigned to the best and worst individuals, respectively. $i \in \{1, 2, 3, \ldots, NP\}$ denotes the index in a sorted fitness array for the population. k is the control factor for adjusting the greediness of rank-based selection [20].

Moreover, F_i and Cr_i generated using Eq. (13) and Eq.(22) during the evolution are tunned as follows.

$$F_i = \begin{cases} \min(F_i, 0.7), & FEs < 0.6 \cdot FEs_{\max} \\ F_i, & 0.6 \cdot FEs_{\max} \le FEs \end{cases} \tag{33}$$

$$Cr_i = \begin{cases} \max(Cr_i, 0.7), & FEs < 0.25 \cdot FEs_{\max} \\ \max(Cr_i, 0.6), & 0.25 \cdot FEs_{\max} \le FEs < 0.5 \cdot FEs_{\max} \\ Cr_i, & 0.5 \cdot FEs_{\max} \le FEs \end{cases} \tag{34}$$

Finally, the parameter settings in LSHADE-RSP that differ from those in jSO are as follows: $p_{\max}$ in Eq.(27) is set to 0.17, k in Eq.(32) is set to 3, and all entries in $\boldsymbol{M}_F$ are initialized to 0.3.

3. The proposed APSM-jSO algorithm

A series of work have been made to DE and its variants in terms of adaptive control parameter setting and modification of the mutation strategy during the past two decades. The most competitive variant of DE is jSO, proposed by Brest in 2017, which integrates the effective improvements for DE proposed in recent years, and it was the winner of the IEEE CEC 2017 competition [17]. However, the adaptation mechanism in jSO may not fully exploit its optimization potential. In this paper, a jSO-based algorithm named APSM-jSO is developed by making simple and effective modifications to the scaling factor selection, crossover rate selection, and external archive update mechanism to fully exploit its optimization performance. The details of the proposed APSM-jSO are as follows.

3.1. Adaptive parameter selection mechanism (APSM)

In the framework of jSO, the scaling factor F and the crossover rate Cr play an important role on its performance [17]. Furthermore, the successful historical memory $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_{\text{F}}$ have significantly improved the effectiveness of jSO. However, better entries in the historical memory are not used more frequently to accelerate evolutionary processes because entries in $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_{\text{F}}$ are randomly selected for use in the existing SHADE variants. To fully utilize better entries in historical memory, a novel adaptive parameter selection mechanism is proposed for selecting entries in $\boldsymbol{M}_{\text{F}}$ and $\boldsymbol{M}_{\text{CR}}$ to generate more suitable F_i and Cr_i. In this mechanism, different probabilities are assigned entries in the $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_{\text{F}}$ according to the historically used information in the last generation. Specifically, the selected probability of entry h in $\boldsymbol{M}_{CR}$ ($\boldsymbol{M}_{\text{F}}$) is calculated at each generation, as shown in Eq. (35).

$$PR_h = \frac{SR_h}{\sum_{h=1}^{H} SR_h}, h = 1, 2, \cdots, H \tag{35}$$

$$SR_h = \frac{SN_h}{N_h}, h = 1, 2, \cdots, H \tag{36}$$

where N_h denotes the number of calls of entry h in $\boldsymbol{M}_{CR}$ ($\boldsymbol{M}_{\mathrm{F}}$) for generating F_i and Cr_i. SN_h denotes the number of entry h generating the successful F_i and Cr_i, which indicate that $\boldsymbol{u}_i$ generated using them is better than $\boldsymbol{x}_i$. Moreover, the successful rate SR_h for the newly updated entry in the $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_F$ is set to be equal to max ($\boldsymbol{SR}$). Moreover, all probabilities for the entries in $\boldsymbol{M}_{CR}$ and $\boldsymbol{M}_F$ will be reset to $1/H$ if all trial individuals are worse than the corresponding target individuals. It is noted that only the entries in $\boldsymbol{M}_F$ and $\boldsymbol{M}_{CR}$ from 1 to H-1 are updated during the evolution in the paper.

3.2. First in-first out method (FIFO)

In the framework of jSO, if the external archive size $|\boldsymbol{A}|=NP\times A_{\mathrm{rate}}$ reaches the predefined size, a random individual will be discarded to make room for the newly inserted individual. Some individuals in the external archive may not be discarded for many generations due to the random update mechanism, which lead to the overuse of some elements in the external archive especially in the later stages of optimization. To avoid overuse of $\boldsymbol{A}$ and enrich population diversity, the first in-first out (FIFO) method is introduced to APSM-jSO to update the external archive if its size reaches the predefined size. In FIFO, the newly inserted individual will replace the earliest preserved individual if the archive size $|\boldsymbol{A}|$ reaches the predefined size.

3.3. New mutation strategy based on RSP

To enhance the exploitation of APSM-jSO, the RSP strategy in LSHADE-RSP as described in Eqs. (31)–(32) are used in APSM-jSO to proposed a new RSP-based mutation strategy. Different from LSHADE-RSP, the RSP is only utilized for r_1 in APSM-jSO instead of both r_1 and r_2 as described in Eq.(30). The advantage of random selection of $\boldsymbol{x}_{r2}$ from the combination of $\boldsymbol{A}$ and $\boldsymbol{P}$ is that the external archive mechanism can give full play to its role, especially at the later stage of evolution, which can effectively improve the population diversity. Moreover, the rank-based selective pressure strategy leads to $\boldsymbol{x}_{pr1}$ higher probability of being superior to $\boldsymbol{x}_{r2}$, that is, the evolution direction of the population has a higher probability of evolving in a better direction, as shown in Fig. 1(a). Whereas it is illustrated in Fig. 1 (b).

Therefore, the new mutation strategy based on RSP is described in Eq.(37).

$$\boldsymbol{v}_i = \boldsymbol{x}_i + Fw_i \cdot \left(\boldsymbol{x}_{pbest} - \boldsymbol{x}_i\right) + F_i \cdot \left(\boldsymbol{x}_{pr_1} - \boldsymbol{x}_{r_2}\right) \tag{37}$$

where $\boldsymbol{x}_{r2}$ is a random individual selected from the combination of $\boldsymbol{A}$ and $\boldsymbol{P}$ that differs from $\boldsymbol{x}_i$, $\boldsymbol{x}_{pbest}$, or $\boldsymbol{x}_{pr1}$.

In summary, the pseudo code of APSM-jSO is described in Fig. 2. .

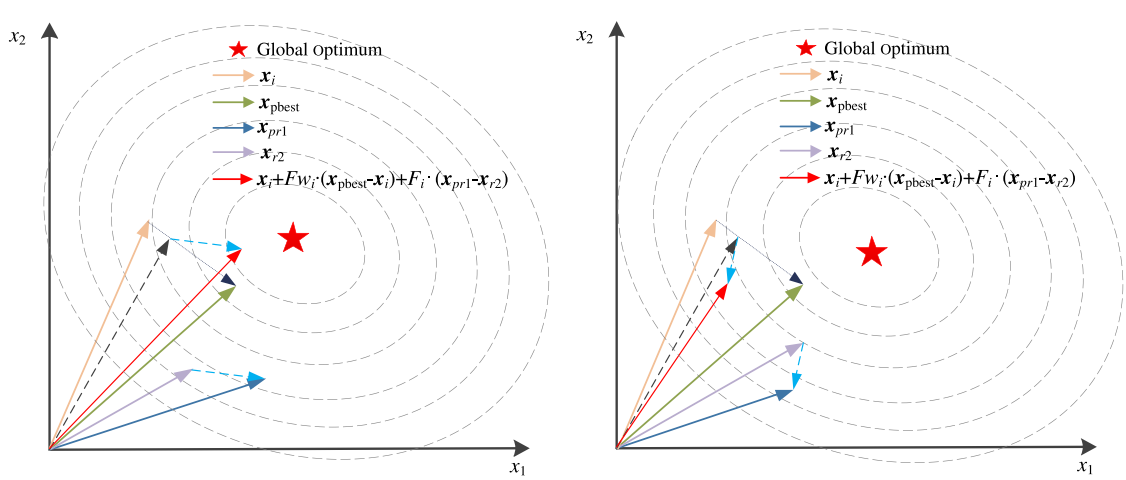
4. Numerical experiments based on CEC 2018 test suite

This section is devoted to investigating the performance of APSM-jSO based on the IEEE CEC 2018 test suite [39] in 10, 30, 50, and 100 dimensions. According to the guideline of the IEEE CEC 2018 test suite [39], the search boundaries are $[-100,100]^D$ and the $FEs_{\max}$ is set to 10,000•D for all benchmark functions, i.e., $FEs_{\max}$ is set to 100,000, 300,000, 500,000, and 1,000,000 for D = 10, 30, 50, and 100, respectively. All numerical experimental results without special instructions were obtained by 51 independent runs. Details of the IEEE CEC 2018 test suite are available in [39]. All the experiments are implemented in MATLAB R2019b on a computer with Inter(R) Core (TM) i7–10750H CPU@2.60 GHz 16GB RAM.

The error fitness value between $\boldsymbol{x}_{Best}$ and $\boldsymbol{x}^*$ defined as $f(\boldsymbol{x}_{Best})$-$f(\boldsymbol{x}^*)$ is recorded as the result. where $f(\boldsymbol{x})$ denotes the objective function. $\boldsymbol{x}^*$ and $\boldsymbol{x}_{Best}$ are the global optimum solution for the problem and the best solution obtained by the algorithm, respectively. The nonparametric tests called Wilcoxon signed rank test and the Friedman test with Iman-Davenport post hoc test [40] are utilized for the comparative analysis in this paper. The Wilcoxon signed rank test utilized to test the statistical significance of two algorithms for each benchmark function based on the results obtained by 51 runs independently, while the Friedman test with Iman-Davenport post hoc test is adopted to further analyze the magnitude of differences [11].

4.1. Parameter tuning

The parameter settings show an important impact on the performance of EAs [30]. Generally, algorithm studies will provide reference values for parameters involved, and these values have been tuned by the authors. In APSM-jSO, the archive size $|\boldsymbol{A}|=NP\times A_{\mathrm{rate}}$ may have a different effect due to the introduction of FIFO. Similarly, the memory size H may also have a different influence due to the introduction of APSM. Therefore, A_{rate} and H are tuned in the section to fully exploit the performance of APSM-jSO. Specifically, A_{rate} is set to {1,1.1,1.2,1.3,1.4,1.5,1.6,1.7,1.8,1.9,2.0}, and H is set to {3,4,5,6,7,8,9,10} in this section.



(a) $\boldsymbol{x}_{pr1}$ is superior to $\boldsymbol{x}_{r2}$. (b) $\boldsymbol{x}_{pr1}$ is inferior to $\boldsymbol{x}_{r2}$.

Fig. 1. Diagram of the new RSP-based mutation strategy in a 2D space.

Algorithm 1: APSM-jSO algorithm.

Input: $f(\boldsymbol{x})$, $FEs_{\max}$, and solution space.
Output: Best solution and objective value to $f(\boldsymbol{x})$.

```
NP=NP_init, NP_min=4, FEs=0, H=6, p_max=0.17, p_min=0.085, A=∅, nA=0, A_rate=1.3, M_CR=0.8, M_F=0.3, PR=1/H, M_F,H=0.9, M_CR,H=0.9.
Initializing population P using Eq.(1).
Evaluating P to determine their fitness value by using f(x).
FEs←FEs+NP.
While FEs< FEs_max do
  Generating p as per Eq.(27).
  Generating Pr as per Eq.(31).
  S_CR =∅, S_F= ∅.
  For i=1 to NP do
    Generating F_i, as per Eq.(13) and Eq.(33).
    Generating Fw_i as per Eq.(29).
    Generating Cr_i as per Eq.(22) and Eq.(34).
    Generating mutant vector v_i using Eq.(37).
    Checking bound-constraints of v_i using Eq.(6).
    Generating u_i using Eq.(7).
    Evaluating u_i to determine its fitness value using f(x).
    FEs=FEs+1.
  End for
  For i=1 to NP do
    If f(u_i)<f(x_i) then
      The FIFO method is adopted to update the external archive A.
      nA=nA+1.
      x_i = u_i.
      f(x_i) = f(u_i).
      S_CR = Cr_i ∪ S_CR.
      S_F =F_i ∪ S_F.
    End if
  End for
  Generating PR as per Eq.(35).
  Updating memory M_F and M_CR as per Eq.(23) and Eq.(24).
  Calculating NP according to Eq.(20).
  Shrinking P by discarding the worst solutions.
  Shrinking A using the FIFO method if necessary.
End while
```

Fig. 2. Pseudo-code for the proposed APSM-jSO.

The other parameters settings follow the values recommended in jSO.

In order to avoid parameter overfitting, only two multimodal functions (F7 and F10), two hybrid functions (F12 and F16), and two composition functions (F24 and F25) included in CEC 2018 test set are used for parameter tuning in this section. All parameters except A_{rate} and H are set as follows: $NP_{\text{init}}=75\bullet D^{2/3}$, $NP_{\min}=4$, $k = 3$, $p_{max}=0.17$, $p_{min}=0.085$. Note that all experimental results were obtained by 5 independent runs in this section, and the average values of the results derived from APSM-jSO with different A_{rate} and H are provided in Tables S1-S4 as the **Supplementary file**, in which the best values are highlighted in **bold**. The Friedman test with a significance level α=0.05 is used to statistically analyze the results in Tables S1-S4. The ranking of APSM-jSO with different A_{rate} and H for all dimensions derived from the Friedman test are depicted in Fig. 3.

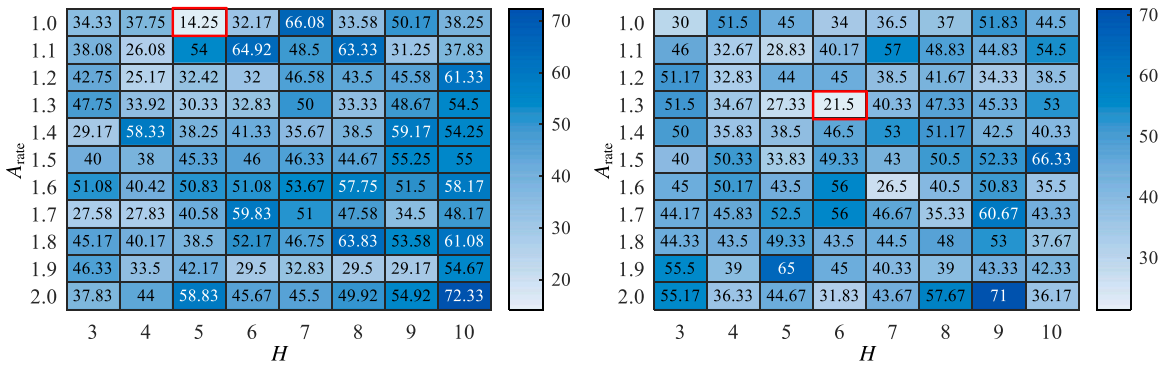
From Fig. 3, APSM-jSO with A_{rate}=1.0 and $H = 5$ obtained the best ranking on 10D in the six functions. For 30D, 50D, and 100D, the optimal values of A_{rate} and H are set as 1.3 and 6, 1.3 and 6, 1.5 and 4, respectively. Fig. 3 (e) shows the average rankings of all dimensions, and the optimal values of A_{rate} and H are set as 1.3 and 6, i.e., APSM-jSO with A_{rate}=1.3 and $H = 6$ shows the best overall performance on all dimensions.

In summary, A_{rate}=1.3 and $H = 6$ are recommended for our proposed APSM-jSO to fully exploit its overall performance, and the values are used in all subsequent experiments in this paper. It should be noted that this setting is recommended under the condition of avoiding overfitting. When solving practical engineering problems, it can be further tuned based on the experiment in this section according to the characteristics of the problem.

4.2. Strategy effectiveness analysis

In this paper, three modifications are made for jSO to enhance its performance. Four APSM-jSO variants are designed to investigate the influence of APSM, FIFO, and RSP on the performance of APSM-jSO. Details of APSM-jSO and its four variants, named APSM-jSO1, APSM-jSO2, APSM-jSO3, and APSM-jSO4 are shown in Table 2. The parameter settings for four APSM-jSO variants are consistent with those of APSM-jSO, and the parameter settings for jSO are set as in the original work, which are provided in Table 4. Details of the results obtained by APSM-jSO, APSM-jSO1, APSM-jSO2, APSM-jSO3, APSM-jSO4, and jSO [17] for the IEEE CEC 2018 test suite are provided in Table.S5-Table.S8 as a **Supplementary file**.

To comprehensively analyze the influence of APSM, FIFO, and RSP on APSM-jSO, Table 3 summarizes the rankings for APSM-jSO, APSM-



(a) Rankings derived from the Friedman test for 10D

(b) Rankings derived from the Friedman test for 30D

(c) Rankings derived from the Friedman test for 50D

(d) Rankings derived from the Friedman test for 100D

(e) Average rankings of all dimensions derived from the Friedman test

Fig. 3. The Friedman test results of APSM-jSO with different A_{rate} and H.

Table 2
Details of APSM-jSO and its four variants.

Strategy	APSM-jSO	APSM-jSO1	APSM-jSO2	APSM-jSO3	APSM-jSO4	jSO
APSM	Yes	Yes	Yes	No	Yes	No
FIFO	Yes	Yes	No	Yes	Yes	No
RSP	Yes	No	Yes	Yes	RSP used in [20]	No

jSO1, APSM-jSO2, APSM-jSO3, APSM-jSO4, and jSO derived from the Friedman test. "Mean" in Table 3 denotes the mean of the rankings of all dimensions, and "Mean Rank" is the sorting result of "Mean".

As shown in Table 3, *p-value* for all dimensions is less than α, which indicates a significant difference in terms of performance among APSM-jSO, APSM-jSO1, APSM-jSO2, APSM-jSO3, APSM-jSO4, and jSO on all dimensions. The Nemenyi test used in our previous work [41] is also used as a post hoc test to further demonstrate the difference among APSM-jSO, APSM-jSO1, APSM-jSO2, APSM-jSO3, APSM-jSO4, and jSO based on the Friedman test results. Fig. 4 presents the magnitude of

Table 3
Rankings of six algorithms obtained by the Friedman test (α=0.05).

Algorithm	10D[a]	30D[b]	50D[c]	100D[d]	Mean	Mean Rank
APSM-jSO	**2.72**	**2.36**	**2.43**	2.97	**2.62**	**1**
APSM-jSO1	3.41	4.26	4.55	4.90	4.28	6
APSM-jSO2	3.19	3.59	3.97	4.17	3.73	4
APSM-jSO3	3.64	3.52	2.47	**2.14**	2.94	2
APSM-jSO4	3.47	2.72	3.64	3.45	3.32	3
jSO	4.57	4.55	3.95	3.38	4.11	5

[a] *p*-value derived from the Friedman test is 1.94E-03, and the *chi-square* is 18.98.
[b] *p*-value derived from the Friedman test is 1.04E-05, and the *chi-square* is 30.78.
[c] *p*-value derived from the Friedman test is 8.63E-06, and the *chi-square* is 31.18.
[d] *p*-value derived from the Friedman test is 4.16E-07, and the *chi-square* is 37.79.

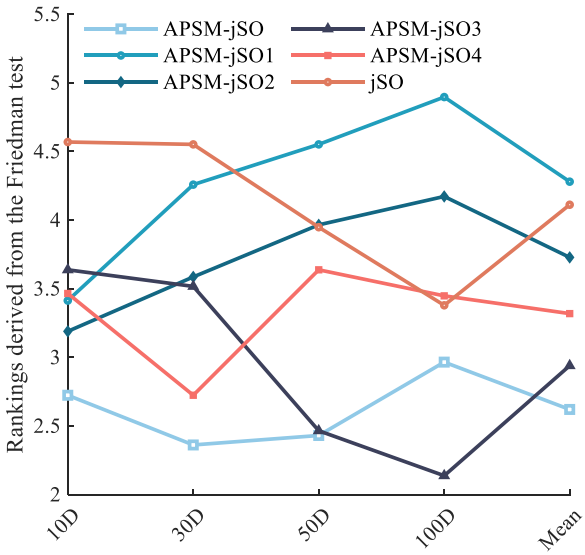


Fig. 4. Rankings of jSO, APSM-jSO, and its variants obtained from the Friedman test (α=0.05).

differences among jSO, APSM-jSO and its four variants, in which the algorithms with no significant difference in terms of performance can be connected using *CDV*. The discussions based on Fig. 4 and Fig. 5 are as follows:

a) The influence of APSM on the performance of APSM-jSO can be investigated by comparing APSM-jSO with APSM-jSO3. APSM-jSO is superior to APSM-jSO3 on 10D, 30D, and 50D, but is inferior to APSM-jSO3 on 100D. That is, APSM can further enhance the performance of APSM-jSO in 10D, 30D, and 50D, but weakens its performance in 100 dimensions.
b) The influence of FIFO on the performance of APSM-jSO can be investigated by comparing APSM-jSO with APSM-jSO2. APSM-jSO is superior to APSM-jSO2 on 10D, 30D, 50D, and 100D. That is, FIFO can further enhance the performance of APSM-jSO in all dimensions.
c) The influence of RSP on the performance of APSM-jSO can be investigated by comparing APSM-jSO with APSM-jSO1 and APSM-jSO4. APSM-jSO is superior to APSM-jSO1 and APSM-jSO4 on all dimensions. That is, the new mutation strategy based on RSP is better than the mutation strategy based on RSP in LSHADE-RSP, and it can further enhance the performance of APSM-jSO in all dimensions.
d) The effectiveness of improvement for APSM-jSO can be investigated by comparing APSM-jSO with jSO: APSM-jSO is superior to jSO on all dimensions. That is, the improvement strategies proposed in our work can effectively enhance the performance of jSO. From Fig. 5, APSM-jSO significantly outperforms jSO on 10D, 30D, and 50D, and is superior to jSO on 100D.

"Mean Rank" is utilized to further assess the effectiveness of improvement strategies on the overall performance of APSM-jSO. From "Mean Rank", APSM-jSO ranks in the first place followed by APSM-jSO3, APSM-jSO4, APSM-jSO2, jSO, and APSM-jSO1. APSM-jSO is superior to jSO and four APSM-jSO variants, which demonstrates that the improvement strategies proposed in our work effectively and significantly enhances the optimization performance of jSO, and APSM-jSO is a promising variant of jSO.

To further analyze the performance of APSM-jSO on a micro-level, the changes in population statistics during the iterative process are discussed. Fig. 6 depicts the standard deviation of every dimension for all individuals when solving unimodal function F1 (30*D*), multimodal function F7 (30*D*), hybrid function F12 (30*D*), and composition function F24 (30*D*) in one run. From Fig. 6, the standard deviation for all dimensions of population converges fast to 0 when solving unimodal function F1, multimodal function F7, and hybrid function F12, but it converges slowly for composition function F24. In summary, the performance APSM-jSO is better on unimodal functions, multimodal functions, and hybrid functions than on composition functions in terms of convergence.

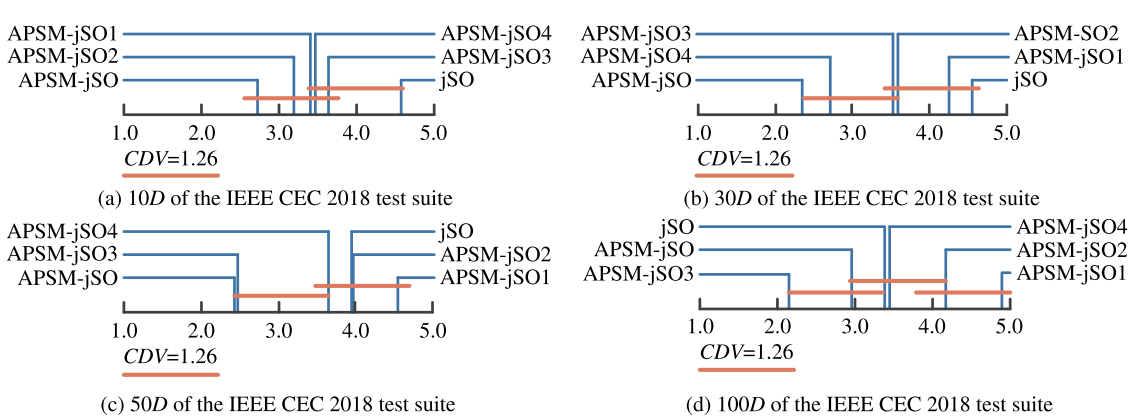
4.3. Comparison with state-of-the-art DE variants

In this section, the performance of APSM-jSO is evaluated using CEC 2018 test suite in comparison with state-of-the-art LSHADE or jSO variants including EB-LSHADE [36], ELSHADE-SPACMA (the winner of the IEEE CEC 2018 competition) [37], LSHADE-RSP (the winner of the IEEE CEC 2018 competition) [20], MadDE (the winner of the IEEE CEC 2021 competition) [38], and mL-SHADE [35]. EB-LSHADE and mL-SHADE, two recently improved versions of L-SHADE, have shown excellent optimization performance. To establish a fair comparison and eliminate disputes, the parameter settings of competitors follow the values recommended in the source literature and are listed in Table 4. Details of the results obtained by APSM-jSO and five DE variants are provided in Tables S9-S12 as the **Supplementary file**.

4.3.1. Analysis for the Wilcoxon signed-rank test results

Table 5 summarizes the Wilcoxon signed rank test results (α=0.05) for the 51 independent results obtained by APSM-jSO, EB-LSHADE, LSHADE-RSP, ELSHADE-SPACMA, mL-SHADE, and MadDE. Details of the Wilcoxon signed rank test results are provided in TableS9-Table.S12 as the **Supplementary file**. In Table 5, the symbols '+', '−', and '=' denote that APSM-jSO show statistically better, worse, and similar performance compared to the competitor, respectively.

The Wilcoxon signed-rank test results are intuitively presented in Fig. 7 to assess algorithm performance from the perspective of test



(a) 10*D* of the IEEE CEC 2018 test suite (b) 30*D* of the IEEE CEC 2018 test suite

(c) 50*D* of the IEEE CEC 2018 test suite (d) 100*D* of the IEEE CEC 2018 test suite

Fig. 5. Multiple comparisons using *CDV* to evaluate jSO, APSM-jSO and its four variants.

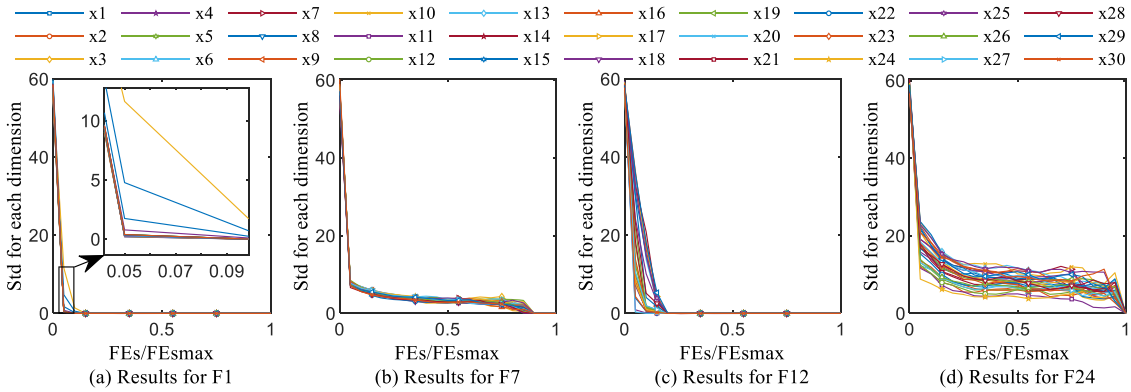


Fig. 6. Standard deviation of every dimension for all individuals.

Table 4
Parameter settings for APSM-jSO and competitors.

Algorithm	Parameter settings
APSM-jSO	NP_{init}=75•$D^{(2/3)}$, $NP_{\min}$=4, k = 3, H = 6, $\lvert\boldsymbol{A}\rvert$=1.3•$NP$, p_{max}=0.17, p_{min}=p_{max}/2.
EB-LSHADE	NP_{init}=18•D, NP_{min}=4, H = 5, c = 0.8, p = 0.11, $\lvert\boldsymbol{A}\rvert$=1.4•$NP$ as in [36].
ELSHADE-SPACMA	NP_{init}=18•D, NP_{min}=4, F_{CP}=0.5, $\lvert\boldsymbol{A}\rvert$=1.4•$NP$, H = 5, c = 0.8, *threshold* =FEs_{max}/2, p_{init}=0.3, p_{min}=0.15, p_{AGDE}=0.1 as in [37].
jSO	NP_{init} =25log(D)$\sqrt{D}$, NP_{min} =4, H = 5, $\lvert\boldsymbol{A}\rvert$=2.6• NP, p_{max} =0.25, p_{min}=0.125 as in [17].
LSHADE-RSP	NP_{init}=75•$D^{(2/3)}$, $NP_{\min}$=4, k = 3, H = 5, $\lvert\boldsymbol{A}\rvert$=$NP$, p_{max}=0.17, p_{min}=0.085.as in [20].
MadDE	NP_{init}=2•D^2, NP_{min}=4, p_{qBX}= 0.01, p = 0.18, $\lvert\boldsymbol{A}\rvert$=2.3•$NP$, H = 10•D, F_0=0.2, and Cr_0=0.2 as in [38].
mL-SHADE	NP_{init}=18•D, NP_{min}=4, p = 0.11, $\lvert\boldsymbol{A}\rvert$=$NP$, H = 6, p_m=1/D, m_r=0.05, η=10, N^{stuck}=400 as in [35].

function dimensions. A detailed analysis is as follows.

a) For 10D, APSM-jSO is better (worse) than ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE on 9(4), 10(5), 4(2), 16(7), and 16(2) test functions. That is, APSM-jSO outperforms five DE-based competitors on 10D.
b) For 30D, APSM-jSO is better (worse) than ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE on 12(5), 10(4), 8 (4), 27(2), and 27(1) test functions. That is, APSM-jSO outperforms five DE-based competitors on 30D.
c) For 50D, APSM-jSO is better (worse) than ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE on 9(10), 15(6), 8 (9), 27(2), and 28(0) test functions. That is, APSM-jSO outperforms EB-LSHADE, MadDE, and mL-SHADE, but is slightly worse than ELSHADE-SPACMA and LSHADE-RSP on 50D.
d) For 100D, APSM-jSO is better (worse) than to ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE on 7(16), 17 (7), 7(10), 29(0), and 27(0) test functions. That is, APSM-jSO outperforms EB-LSHADE, MadDE, and mL-SHADE, but is worse than ELSHADE-SPACMA and LSHADE-RSP on 100D.

The results listed in Table 5 are intuitively presented in Fig. 8 to evaluate the performance of algorithms from the perspective of test function type. A detailed analysis is as follows.

a) For unimodal functions, APSM-jSO is superior (inferior) to ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE on 2(3), 2(0), 3(0), 7(0), and 6(0) test functions. That is, APSM-jSO is superior to EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE, but is slightly worse than ELSHADE-SPACMA on unimodal functions.
b) For simple multimodal functions, APSM-jSO is superior (inferior) to ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE on 10(9), 8(9), 8(7), 27(0), and 23(1) test functions. That is, APSM-jSO outperforms five DE-based competitors on simple multimodal functions.
c) For hybrid functions, APSM-jSO is superior (inferior) to ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE on 9 (12), 25(8), 4(14), 34(2), and 34(2) test functions. That is, APSM-jSO outperforms EB-LSHADE, MadDE, and mL-SHADE, but is worse than ELSHADE-SPACMA and LSHADE-RSP on hybrid functions.
d) For composition functions, APSM-jSO is superior (inferior) to ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE on 16(11), 17(5), 12(4), 31(9), and 35(0) test functions. That is, APSM-jSO outperforms five DE-based competitors on composition functions.

According to "Total" listed in Table 5, APSM-jSO is superior (inferior) to ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, MadDE, and mL-SHADE on 37(35), 52(22), 27(25), 99(11), and 98(3) test functions, i.e., APSM-jSO get more '+' than '-' in comparison with five DE-based variants, which implies that APSM-jSO is superior to five DE variants. In summary, APSM-jSO is better than ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, mL-SHADE, and MadDE in terms of overall performance.

4.3.2. Analysis for the Friedman test results

In this section, the results derived from the Friedman test are analyzed to compare the overall performance differences among APSM-jSO and five DE variants. The rankings obtained by the Friedman test are summarized in Table 6 and intuitively presented in Fig. 9. From Table 6, p-values for 30D, 50D, and 100D are less than α, that indicates some significant difference among APSM-jSO and five DE-based variants. Details of the Friedman test results are as follows:

a) For 10D, APSM-jSO ranks in the first place followed by LSHADE-RSP, EB-LSHADE, ELSHADE-SPACMA, MadDE, and mL-SHADE. That is, APSM-jSO outperforms five DE-based competitors on 10D.
b) For 30D, APSM-jSO ranks in first place followed by LSHADE-RSP, EB-LSHADE, ELSHADE-SPACMA, mL-SHADE, and MadDE. That is, APSM-jSO outperforms five DE-based competitors on 30D.
c) For 50D, APSM-jSO ranks in first place followed by LSHADE-RSP, ELSHADE-SPACMA, EB-LSHADE, mL-SHADE, and MadDE. That is, APSM-jSO outperforms five DE-based competitors on 50D.
d) For 100D, APSM-jSO ranks in second place following ELSHADE-SPACMA, followed by LSHADE-RSP, EB-LSHADE, mL-SHADE, and MadDE. That is, APSM-jSO is superior to five DE-based competitors except ELSHADE-SPACMA on 100D.

Based on the above discussions, APSM-jSO is superior to EB-LSHADE, LSHADE-RSP, mL-SHADE, and MadDE on all dimensions, superior to

Table 5
The Wilcoxon signed-rank test results of APSM-jSO and five DE variants (α=0.05).

R (+/-/=)	Function Type	10D	30D	50D	100D	Total
APSM-jSO vs. EB-LSHADE	Unimodal Functions	0/0/2	0/0/2	0/0/2	2/0/0	2/0/6
	Simple Multimodal Functions	4/0/3	0/2/5	2/4/1	2/3/2	8/9/11
	Hybrid Functions	2/5/3	7/1/2	8/0/2	8/2/0	25/8/7
	Composition Functions	4/0/6	3/1/6	5/2/3	5/2/3	17/5/18
	Total	10/5/14	10/4/15	15/6/8	17/7/5	52/22/42
APSM-jSO vs. ELSHADE-SPACMA	Unimodal Functions	0/0/2	0/1/1	0/2/0	2/0/0	2/3/3
	Simple Multimodal Functions	3/0/4	4/1/2	2/3/2	1/5/1	10/9/9
	Hybrid Functions	2/2/6	4/1/5	3/2/5	0/7/3	9/12/19
	Composition Functions	4/2/4	4/2/4	4/3/3	4/4/2	16/11/13
	Total	9/4/16	12/5/12	9/10/10	7/16/6	37/35/44
APSM-jSO vs. LSHADE-RSP	Unimodal Functions	0/0/2	1/0/1	0/0/2	2/0/0	3/0/5
	Simple Multimodal Functions	2/0/5	2/1/4	3/3/1	1/3/3	8/7/13
	Hybrid Functions	1/2/7	2/3/5	0/4/6	1/5/4	4/14/22
	Composition Functions	1/0/9	3/0/7	5/2/3	3/2/5	12/4/24
	Total	4/2/23	8/4/17	8/9/12	7/10/12	27/25/64
APSM-jSO vs. MadDE	Unimodal Functions	1/0/1	2/0/0	2/0/0	2/0/0	7/0/1
	Simple Multimodal Functions	6/0/1	7/0/0	7/0/0	7/0/0	27/0/1
	Hybrid Functions	4/2/4	10/0/0	10/0/0	10/0/0	34/2/4
	Composition Functions	5/5/0	8/2/0	8/2/0	10/0/0	31/9/0
	Total	16/7/6	27/2/0	27/2/0	29/0/0	99/11/6
APSM-jSO vs. mL-SHADE	Unimodal Functions	0/0/2	2/0/0	2/0/0	2/0/0	6/0/2
	Simple Multimodal Functions	5/0/2	5/1/1	7/0/0	6/0/1	23/1/4
	Hybrid Functions	4/2/4	10/0/0	10/0/0	10/0/0	34/2/4
	Composition Functions	7/0/3	10/0/0	9/0/1	9/0/1	35/0/5
	Total	16/2/11	27/1/1	28/0/1	27/0/2	98/3/15

ELSHADE-SPACMA on 10D, 30D, and 50D, but is inferior to it on 100D. According to the "Mean Rank" in Table 6, APSM-jSO ranks in the first place followed by LSHADE-RSP, ELSHADE-SPACMA, EB-LSHADE, mL-SHADE, and MadDE. It can be concluded that APSM-jSO is superior to all competitors, which demonstrates that APSM-jSO significantly enhances the performance of jSO, and APSM-jSO is a promising variant of jSO.

The Nemenyi test [41] is also used as a post hoc test to further demonstrate the difference among APSM-jSO, EB-LSHADE, ELSHADE-SPACMA, LSHADE-RSP, MadDE, and mL-SHADE based on the Friedman test results. Fig. 10 presents the magnitude of differences among APSM-jSO and five DE-based competitors.

According to Fig. 10, APSM-jSO ranks in the first place and has no significant differences with all competitors on 10D. For 30D, 50D, and 100D, APSM-jSO shows no significant differences between LSHADE-RSP, EB-LSHADE, and ELSHADE-SPACMA, and is significantly superior to mL-SHADE and MadDE. In conclusion, APSM-jSO is a very competitive algorithm.

4.3.3. Analysis of the convergence performance and robustness

In this section, the convergence performance and robustness of APSM-jSO are discussed. For brevity, the convergence graphs and box graphs for only six sampled functions on 30D, one unimodal function (F1), one simple multimodal function (F9), two hybrid functions (F15 and F18), and two composition functions (F22 and F28), are presented in this section. All convergence graphs and box graphs for all functions on all dimensions are presented in Figs. S1-S16 as the **supplementary file**.

The average convergence graph based on 51 independent runs of APSM-jSO and five DE-based competitors for the above-mentioned functions is depicted as Fig. 11. APSM-jSO shows better performance than MadDE for all functions in terms of convergence speed, and APSM-jSO shows no significant difference between ELSHADE-SPACMA, EB-LSHADE, mL-SHADE, and LSHADE-RSP. In conclusion, APSM-jSO is better than MadDE, and shows no significant difference with the other four competitors on the convergence performance.

The box graphs of the results obtained by 51 independent runs are shown as Fig. 12 to demonstrate the robustness of APSM-jSO. The box graph includes a central median value, outliers, 1/4 and 3/4 values in the 51 results. From Fig. 12, APSM-jSO shows greater robustness and stability than MadDE and is similar to ELSHADE-SPACMA, EB-LSHADE, mL-SHADE, and LSHADE-RSP. That is, while enhancing the performance of APSM-jSO, it does not weaken the robustness.

4.3.4. Analysis of computational complexity

In this section, the instruction for evaluating algorithms complexity in the IEEE CEC 2018 test suite [39] is adopted to test the computational complexity of APSM-jSO. The algorithm complexity is reflected by $(\bar{T}_2 - T_1)/T_0$, and the meanings of T_0, T_1, and $\bar{T}_2$ can refer to [39]. In this section, NP_{init} of APSM-jSO and six DE-based variants on all dimensions is set to 500 to make a fair comparison, and other parameters are consistent with those in Table 4.

The computational complexity of APSM-jSO, ELSHADE-SPACMA, jSO, EB-LSHADE, LSHADE-RSP, mL-SHADE, and MadDE are listed in Table 7 and shown as Fig. 13 for a more intuitive comparison. The computational complexity of APSM-jSO is lower than that of jSO, ELSHADE-SPACMA, EB-LSHADE, and MadDE in all dimensions, and is lower than that of mL-SHADE on all dimensions except 100D. The complexity of APSM-jSO is lower than that of LSHADE-RSP in 10D, but is higher on 30D, 50D, and 100D. In conclusion, APSM-jSO performs more efficiently than jSO, ELSHADE-SPACMA, EB-LSHADE, mL-SHADE, and MadDE, and shows similar performance to LSHADE-RSP in terms of computational complexity.

4.4. Comparison with the winners of the IEEE CEC competitions

In order to further evaluate the performance of APSM-jSO, APSM-jSO is evaluated by comparing it with the winners of the IEEE CEC competitions, those winners are considered state of the art EAs, including AGSK [42] (ranked second in the IEEE CEC 2020 competition), APGSK-IMODE [43] (the winner of the IEEE CEC 2021 competition), EBOwithCMAR [44] (ranked first in the IEEE CEC 2017 competition), HSES [45] (ranked first in the IEEE CEC 2018 competition), IMODE [46] (ranked first in the IEEE CEC 2020 competition). To eliminate disputes and establish a fair comparison, the parameter settings for competitors follow the values recommended in the source literature, that are listed in Table 8. Details of the results obtained by APSM-jSO, AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE are provided in Tables S13-S16 as the **Supplementary file**.

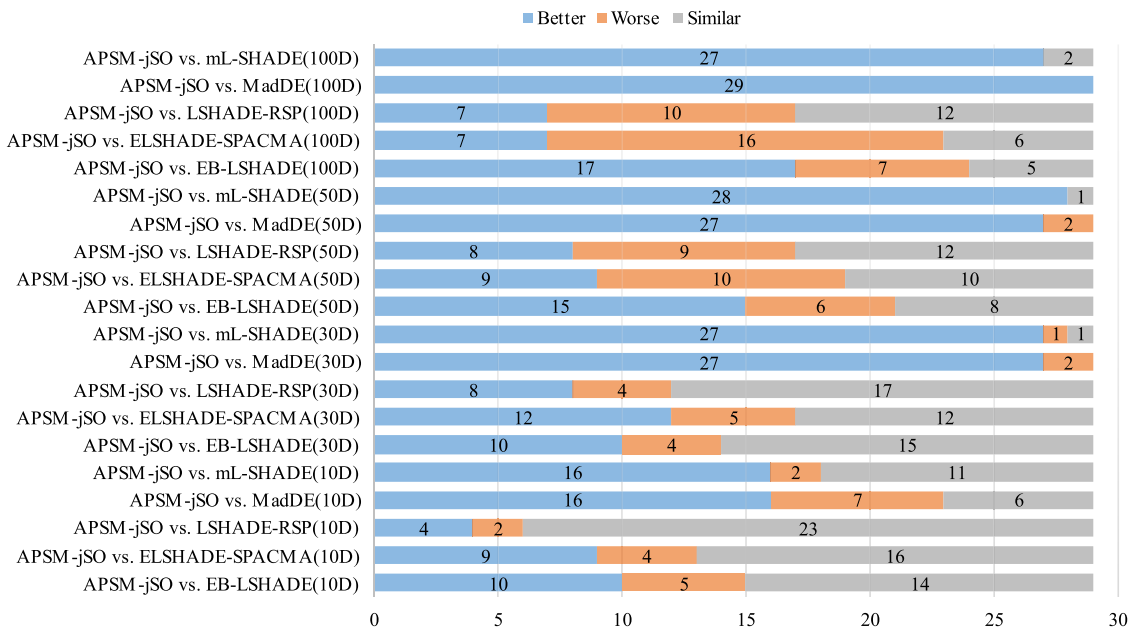


Fig. 7. The Wilcoxon signed-rank test results from the perspective of the test function dimension.

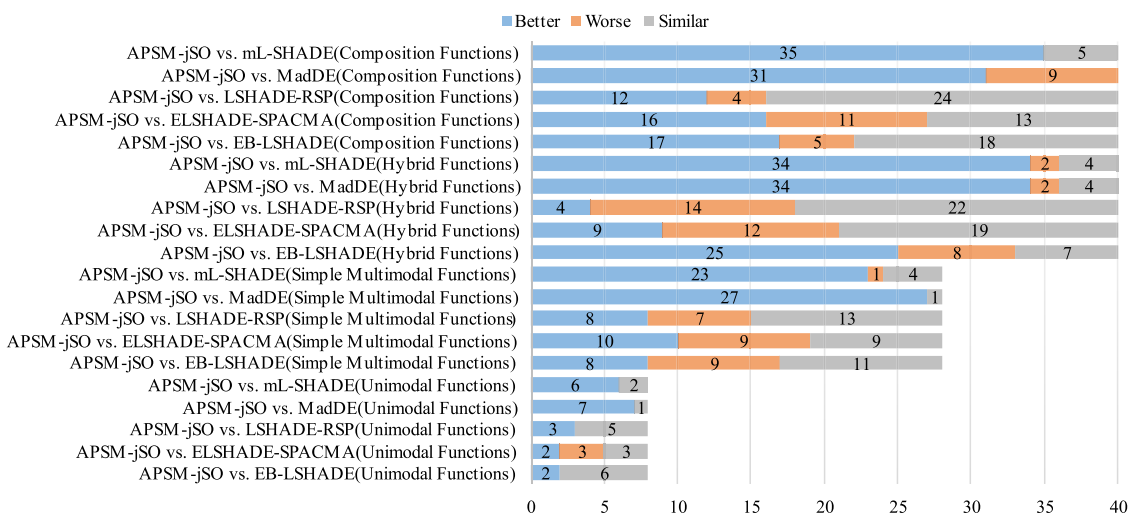


Fig. 8. The Wilcoxon signed-rank test results from the perspective of the test function type.

4.4.1. Analysis for the Wilcoxon signed-rank test results

Table 9 summarizes the Wilcoxon signed rank test (α=0.05) results for the 51 independent results obtained by APSM-jSO, AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE. Details of the Wilcoxon signed rank test results are provided in Tables S13- S16 as the **Supplementary file**.

The results listed in Table 9 are intuitively depicted in Fig. 14 to assess the performance of algorithms from the perspective of test function dimensions. A detailed analysis is as follows.

a) For 10D, APSM-jSO is better (worse) than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 20(6), 17(8), 9(12), 21(3), and 20(7) test functions. That is, APSM-jSO outperforms AGSK, APGSK-IMODE, HSES, and IMODE, but is inferior to EBOwithCMAR on 10D.
b) For 30D, APSM-jSO is better (worse) than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 28(0), 26(1), 12(6), 15(5), and 26(3) test functions. That is, APSM-jSO outperforms AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 30D.
c) For 50D, APSM-jSO is better (worse) than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 29(0), 28(1), 15(9), 8(18), and 27(1) test functions. That is, APSM-jSO outperforms AGSK, APGSK-IMODE, EBOwithCMAR, and IMODE, but is inferior to HSES on 50D.
d) For 100D, APSM-jSO is better (worse) than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 28(1), 29(0), 10(12), 4(21), and 25(4) test functions. That is, APSM-jSO outperforms AGSK, APGSK-IMODE, and IMODE, but is inferior to EBOwithCMAR and HSES on 100D.

The results listed in Table 9 are intuitively presented in Fig. 15 to evaluate the performance of algorithms from the perspective of test function type. A detailed analysis is as follows.

a) For unimodal functions, APSM-jSO is better (worse) than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 7(0), 8(0), 6 (0), 7(0), and 8(0) test functions. That is, APSM-jSO outperforms

Table 6
Rankings obtained by the Friedman test (α=0.05) of APSM-jSO and five DE variants.0

Algorithms	10D[a]	30D[b]	50D[c]	100D[d]	Mean	Mean Rank
APSM-jSO	**3.00**	**2.24**	**2.41**	2.38	**2.51**	**1**
EB-LSHADE	3.45	2.83	2.93	3.03	3.06	4
ELSHADE-SPACMA	3.53	3.10	2.55	**2.00**	2.80	3
LSHADE-RSP	3.05	2.59	2.52	2.76	2.73	2
MadDE	3.74	5.62	5.66	6.00	5.25	6
mL-SHADE	4.22	4.62	4.93	4.83	4.65	5

[a] The *p*-value derived from the Friedman test is 8.61E-02, and the *chi-square* is 9.64.
[b] The *p*-value derived from the Friedman test is 1.01E-14, and the *chi-square* is 74.83.
[c] The *p*-value derived from the Friedman test is 1.66E-16, and the *chi-square* is 83.36.
[d] The *p*-value derived from the Friedman test is 2.22E-20, and the *chi-square* is 101.79.

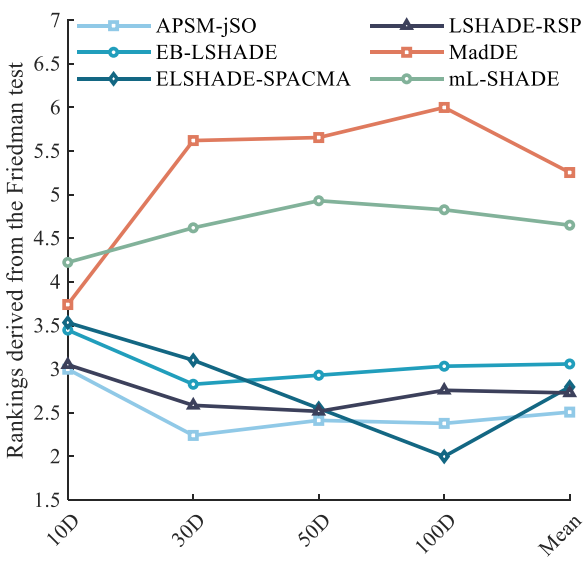


Fig. 9. Rankings of APSM-jSO and five DE variants.

AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on unimodal functions.

b) For simple multimodal functions, APSM-jSO better (worse) than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 28(0), 26(0), 4(16), 5(17), and 25(3) test functions. That is, APSM-jSO outperforms AGSK, APGSK-IMODE, and IMODE, but is inferior to EBOwithCMAR and HSES on multimodal functions.

c) For hybrid functions, APSM-jSO better (worse) than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 38(2), 36(2), 21(8), 16(19), and 37(2) test functions. That is, APSM-jSO outperforms AGSK, APGSK-IMODE, EBOwithCMAR, and IMODE, but is slightly inferior to HSES on hybrid functions.

d) For composition functions, APSM-jSO is better (worse) than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 32(5), 30(8), 15(15), 20(11), and 28(10) test functions. That is, APSM-jSO outperforms AGSK, APGSK-IMODE, HSES, and IMODE, and shows similar performance with EBOwithCMAR on composition functions.

According to "Total" listed in Table 9, APSM-jSO is better (worse) than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE on 105 (7), 100(10), 46(39), 48(47), and 98(15) test functions, i.e., APSM-jSO get more '+' than '-' in comparison with five winners of the IEEE CEC competitions, which implies that APSM-jSO outperforms the five winners of the IEEE CEC competitions. It can be concluded that APSM-jSO is better than AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE in terms of overall performance according to the Wilcoxon signed-rank test results.

4.4.2. Analysis for the Friedman test results

The rankings of APSM-jSO, AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE obtained by the Friedman test (α=0.05) are tabulated as Table 10 and intuitively presented as Fig. 16. *p*-values for 10D, 30D, 50D, and 100D derived from the Friedman test are less than α, that indicates some significant difference among APSM-jSO and five winners of the IEEE CEC competitions. According to the "Mean Rank" listed in the last column of Table 10, APSM-jSO ranks in the second place following in EBOwithCMAR, followed by HSES, APGSK-IMODE, AGSK, and IMODE. The detailed analysis is as follows.

a) For 10D, APSM-jSO ranks in the second place following in EBOwithCMAR, followed by APGSK-IMODE, IMODE, AGSK, and HSES. That is, APSM-jSO is superior to IMODE, AGSK, APGSK-IMODE, and HSES, but inferior to EBOwithCMAR on 10D.
b) For 30D, APSM-jSO ranks in the first place followed by EBOwithCMAR, HSES, APGSK-IMODE, AGSK, and IMODE. That is, APSM-jSO outperforms five winners of IEEE CEC competitions on 30D.
c) For 50D, APSM-jSO ranks in the second place following in HSES, followed by EBOwithCMAR, AGSK, APGSK-IMODE, and IMODE. That is, APSM-jSO is superior to five winners of the IEEE CEC competitions except HSES on 50D.
d) For 100D, APSM-jSO ranks in the third place following HSES and EBOwithCMAR, followed by APGSK-IMODE, AGSK, and IMODE. That is, APSM-jSO is superior to APGSK-IMODE, AGSK, and IMODE, but is inferior to HSES and EBOwithCMAR on 100D.

Based on the above discussions, APSM-jSO is superior to HSES on

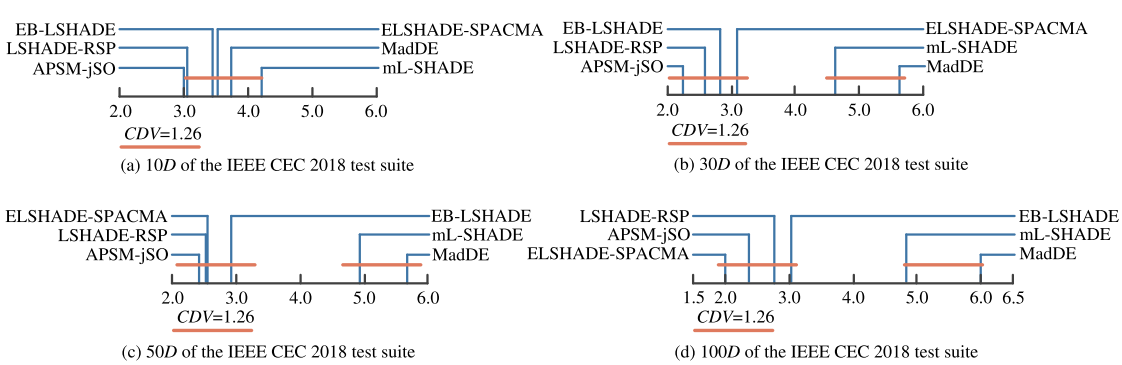


Fig. 10. Multiple comparisons using *CDV* to evaluate APSM-jSO and five DE-based competitors.

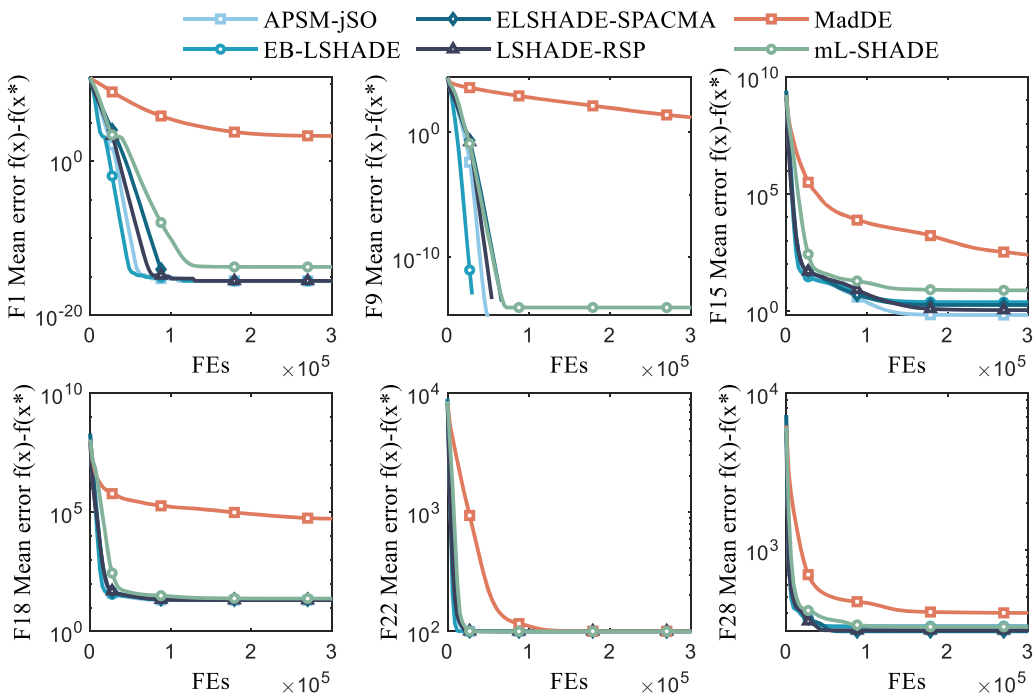


Fig. 11. Convergence graph of APSM-jSO and five DE-based competitors ($D = 30$).

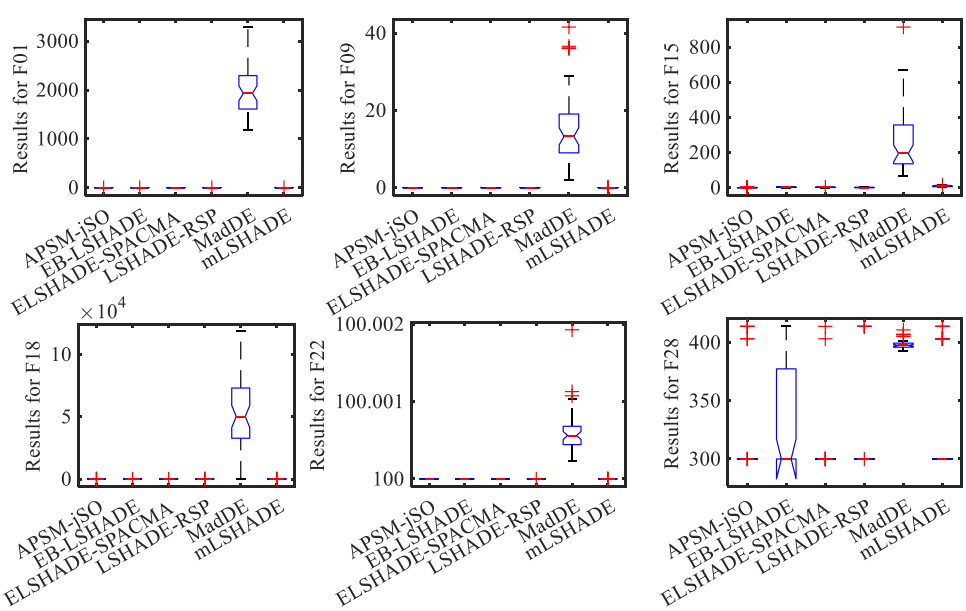


Fig. 12. Box graph of results obtained by APSM-jSO five DE-based competitors ($D = 30$).

Table 7
Computational complexity of APSM-jSO and six DE-based competitors.

Algorithms	$\bar{T}_2/s$				$(\bar{T}_2 - T_1)/T_0$			
	10D	30D	50D	100D	10D	30D	50D	100D
APSM-jSO	0.3456	0.6841	1.1559	3.0301	**25.4856**	36.9170	50.1415	83.2115
EB-LSHADE	0.8882	1.2543	1.8081	3.9783	76.2260	90.2324	111.1259	171.8761
ELSHADE-SPACMA	0.6620	1.0863	1.7869	4.6642	55.0763	74.5251	109.1471	236.0063
jSO	0.6164	0.9219	1.4206	3.3531	50.8121	59.1464	74.8927	113.4197
LSHADE-RSP	0.3943	0.6619	1.1217	2.9546	30.0457	**34.8434**	**46.9449**	**76.1605**
MadDE	0.6660	1.1248	1.7133	3.9179	55.4486	78.1177	102.2579	166.2211
mL-SHADE	0.5405	0.8496	1.2569	3.0069	43.7087	52.3850	59.5930	81.0467

T_0=0.0107 s, T_1=0.0730 s (10D), T_1=0.2893s (30D), T_1=0.6196 s (50D). T_1=2.1401s (100D).

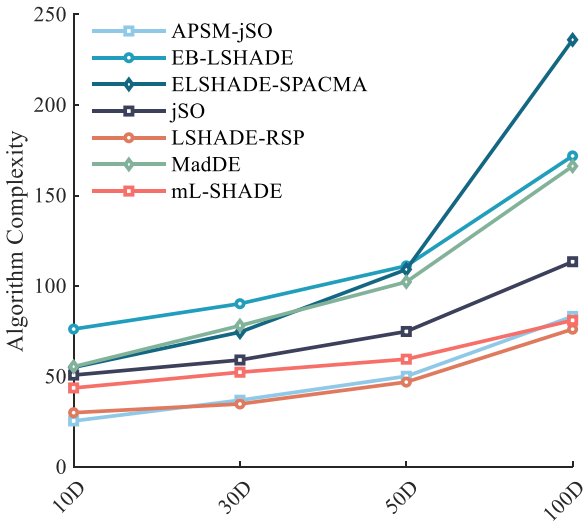


Fig. 13. Computational complexity of APSM-jSO and six DE-based competitors.

Table 8
Parameter settings for APSM-jSO, AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE.

Algorithm	Parameter settings
APSM-jSO	$NP_{\text{init}}=75\bullet D^{(2/3)}$, $NP_{\min}=4$, $k=3$, $H=6$, $\lvert\boldsymbol{A}\rvert=1.3\bullet NP$, $p_{max}=0.17$, $p_{min}=p_{max}/2$.
AGSK	$NP_{max}=20\bullet D$, $NP_{min}=12$, $p=0.05$, $c=0.05$ as in [42].
APGSK-IMODE	$PS_2=7.5\bullet D$, $PS_2^{\min}=12$, $p=0.05$, $c=0.05$, $Kw_P=$ [0.85,0.05,0.05,0,05], $A_{\text{rate}}=1.4$, $H=15\bullet D$, $PS_1=30\bullet D$- PS_2, $NP_1^{\min}=4$, $p^{\text{init}}=0.3$, $p^{\min}=0.15$, $CS=50$ as in [43].
EBOwithCMAR	For EBO, $PS_{1,\ \max}=18\bullet D$, $PS_{1,\ \min}=4$, $H=6$. For CMAR, $\sigma=0.3$, $PS_3=\text{round}(4+3\bullet\log(D))$, $CS=100$, 200, and 300 for the10D, 30D, and 50D respectively. For local search, $prob_{1s}=0.1$ and $cfe_{1s}=0.25\bullet FEs_{\max}$ as in [44].
HSES	$\lambda=\text{round}(3\bullet\ln D)+80$, at first step: $N=100$, $M=200$, $I=20$ and $cc=0.96$, at fourth step: $M=200$, $N=160$ ($D=10$, 30); $N=360$, $M=450$ ($D=50$) as in [45].
IMODE	$NP_{init}=6\bullet D^2$, $NP_{min}=4$, $H=20\bullet D$, $\lvert\boldsymbol{A}\rvert=2.6\bullet NP$, $FE_{LS}=0.85\bullet FEs_{\max}$, $prob_{1s}=0.1$ as in [46].

10D and 30D (low dimensions), but inferior to it on 50D and 100D (high dimensions). It can be concluded that the comprehensive performance of APSM-jSO is comparable to that of EBOwithCMAR, better than that of HSES, and significantly better than that of APGSK-IMODE, AGSK, and IMODE.

The results of the Nemenyi test are depicted in Fig. 17 to further demonstrate the significant difference among APSM-jSO, AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE. From Fig. 17, APSM-jSO, APGSK-IMODE, and EBOwithCMAR do not show significant differences, but they are significantly better than AGSK and HSES on 10D. As for 30D, 50D, and 100D, APSM-jSO does not show significant difference with EBOwithCMAR and HSES, but is significantly better than APGSK-IMODE, AGSK, and IMODE. It can be concluded that APSM-jSO is comparable to that of EBOwithCMAR and HSES, and significantly better than APGSK-IMODE, AGSK, and IMODE. And APSM-jSO is also a top method when solving the IEEE CEC 2018 test suite.

4.4.3. Analysis of the convergence performance and robustness

The convergence performance and robustness of APSM-jSO are further discussed by comparing it with five winners of the IEEE CEC competitions. For brevity, the convergence graphs and box graphs for the same sampled functions as in Section 4.3.3 are presented in this section. All convergence graphs and box graphs for all functions on all dimensions are presented in Figs. S17-S32 as the **supplementary file**.

The average convergence graph based on 51 independent runs of

Table 9
The Wilcoxon signed-rank test results of APSM-jSO and five winners of the IEEE CEC competitions (α=0.05).

R (+/-/=)	Function Type	10D	30D	50D	100D	Total
APSM-jSO vs. AGSK	Unimodal Functions	1/0/1	2/0/0	2/0/0	2/0/0	7/0/1
	Simple Multimodal Functions	7/0/0	7/0/0	7/0/0	7/0/0	28/0/0
	Hybrid Functions	8/2/0	10/0/0	10/0/0	10/0/0	38/2/0
	Composition Functions	4/4/2	9/0/1	10/0/0	9/1/0	32/5/3
	Total	20/6/3	28/0/1	29/0/0	28/1/0	105/7/4
APSM-jSO vs. APGSK-IMODE	Unimodal Functions	2/0/0	2/0/0	2/0/0	2/0/0	8/0/0
	Simple Multimodal Functions	6/0/1	6/0/1	7/0/0	7/0/0	26/0/2
	Hybrid Functions	6/2/2	10/0/0	10/0/0	10/0/0	36/2/2
	Composition Functions	3/6/1	8/1/1	9/1/0	10/0/0	30/8/2
	Total	17/8/4	26/1/2	28/1/0	29/0/0	100/10/6
APSM-jSO vs. EBOwithCMAR	Unimodal Functions	0/0/2	2/0/0	2/0/0	2/0/0	6/0/2
	Simple Multimodal Functions	1/3/3	1/3/3	1/5/1	1/5/1	4/16/8
	Hybrid Functions	5/5/0	6/0/4	7/0/3	3/3/4	21/8/11
	Composition Functions	3/4/3	3/3/4	5/4/1	4/4/2	15/15/10
	Total	9/12/8	12/6/11	15/9/5	10/12/7	46/39/31
APSM-jSO vs. HSES	Unimodal Functions	2/0/0	2/0/0	2/0/0	1/0/1	7/0/1
	Simple Multimodal Functions	4/3/0	0/2/5	0/6/1	1/6/0	5/17/6
	Hybrid Functions	8/0/2	6/2/2	2/7/1	0/10/0	16/19/5
	Composition Functions	7/0/3	7/1/2	4/5/1	2/5/3	20/11/9
	Total	21/3/5	15/5/9	8/18/3	4/21/4	48/47/21
APSM-jSO vs. IMODE	Unimodal Functions	2/0/0	2/0/0	2/0/0	2/0/0	8/0/0
	Simple Multimodal Functions	7/0/0	6/1/0	6/1/0	6/1/0	25/3/0
	Hybrid Functions	8/1/1	10/0/0	10/0/0	9/1/0	37/2/1
	Composition Functions	3/6/1	8/2/0	9/0/1	8/2/0	28/10/2
	Total	20/7/2	26/3/0	27/1/1	25/4/0	98/15/3

APSM-jSO, AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE for the six functions ($D=30$) are shown as Fig. 18. In terms of convergence speed, APSM-jSO shows better performance than AGSK, APGSK-IMODE, and IMODE for all functions, and does not show significant difference with EBOwithCMAR, and HSES. In conclusion, APSM-jSO shows no significant differences with EBOwithCMAR and HSES, and is better than AGSK, APGSK-IMODE, and IMODE in terms of the convergence speed.

The box graphs of the results are shown as Fig. 19 to further demonstrate the robustness of APSM-jSO by comparing it with five winners of the IEEE CEC competitions. From Fig. 19, APSM-jSO shows greater robustness and stability than APGSK-IMODE and IMODE, and is

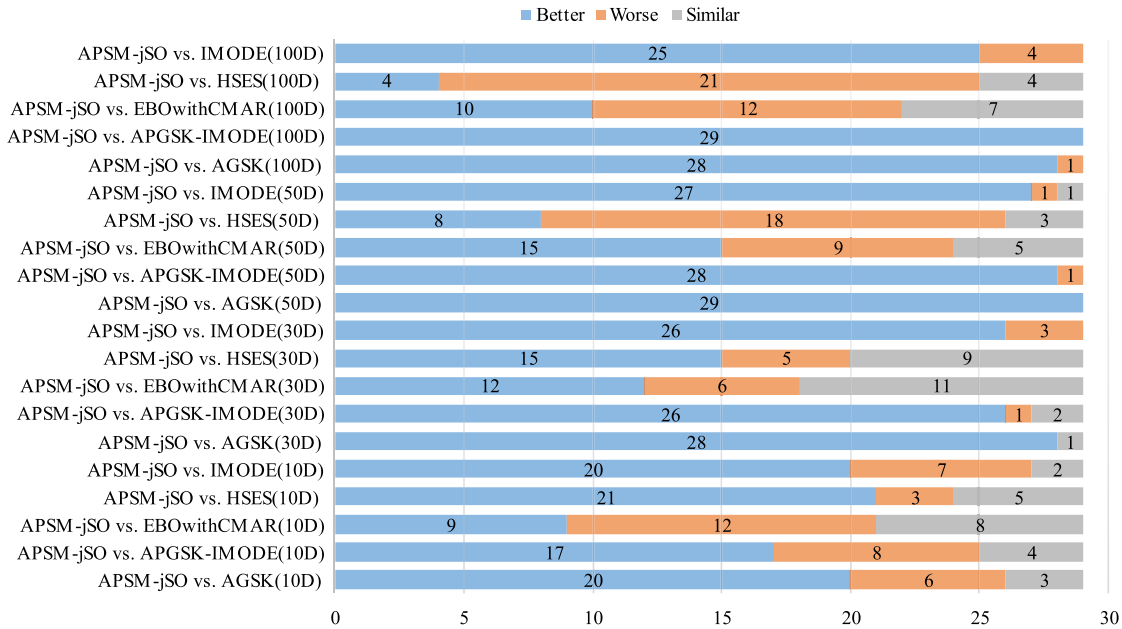


Fig. 14. The Wilcoxon signed-rank test results from the perspective of the test function dimension.

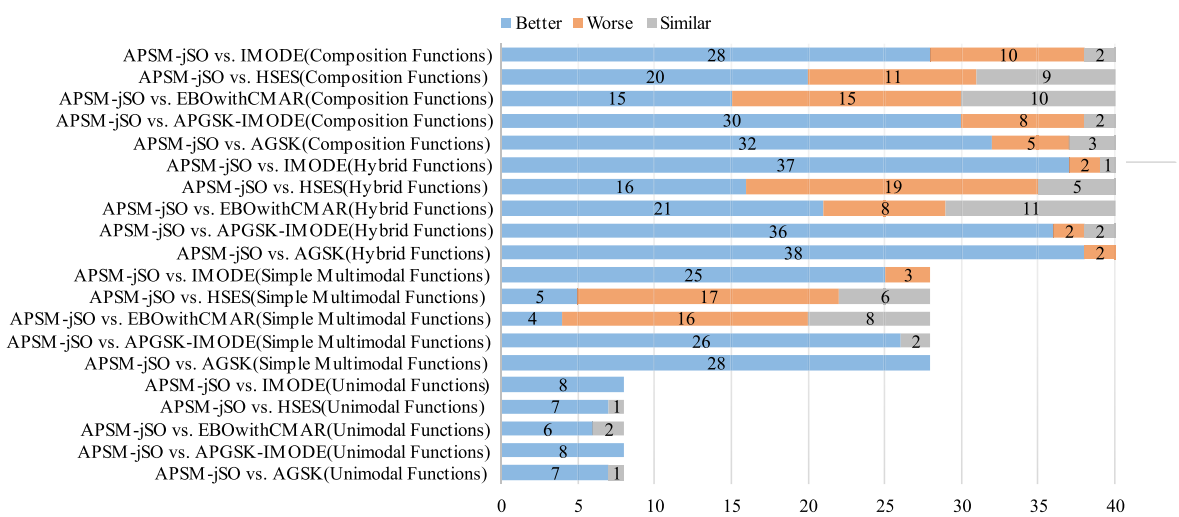


Fig. 15. The Wilcoxon signed-rank test results from the perspective of the test function type.

Table 10
Rankings of APSM-jSO and five winners of the IEEE CEC competitions obtained by the Friedman test (α=0.05).

Algorithm	10D[a]	30D[b]	50D[c]	100D[d]	Mean	Mean Rank
APSM-jSO	2.71	**1.93**	2.10	2.52	2.31	2
AGSK	4.07	4.62	4.69	4.93	4.58	5
APGSK-IMODE	3.00	4.48	4.72	4.79	4.25	4
EBOwithCMAR	**2.53**	1.98	2.34	2.34	**2.30**	1
HSES	4.66	2.81	**1.97**	**1.48**	2.73	3
IMODE	4.03	5.17	5.17	4.93	4.83	6

[a] The *p*-value derived from the Friedman test is 7.62E-06, and the *chi-square* is 31.45.
[b] The *p*-value derived from the Friedman test is 5.91E-17, and the *chi-square* is 85.50.
[c] The *p*-value derived from the Friedman test is 9.49E-19, and the *chi-square* is 94.04.
[d] The *p*-value derived from the Friedman test is 4.01E-20, and the *chi-square* is 100.57.

similar to AGSK, EBOwithCMAR, and HSES. That is, APSM-jSO is comparable to AGSK, EBOwithCMAR, and HSES, and better than APGSK-IMODE and IMODE in terms of robustness.

4.4.4. *Analysis of computational complexity*

The complexity of APSM-jSO is further discussed by comparing five winners of IEEE CEC competitions based on the instructions used in Section 4.3.4. The complexities of APSM-jSO, AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE are listed in Table 11 and intuitively presented as Fig. 20. From Fig. 20, it can be concluded that APSM-jSO performs more efficiently than APSM-jSO, AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE in terms of the computational complexity.

5. Conclusions and future work

In this work, a novel variant of jSO named APSM-jSO is proposed to further enhance the performance of jSO by introducing a novel adaptive

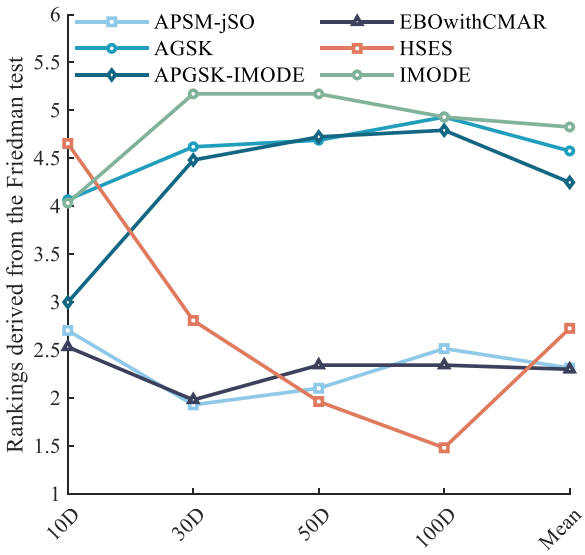


Fig. 16. Rankings of APSM-jSO and five winners of the IEEE CEC competitions obtained by the Friedman test (α=0.05).

selection mechanism (APSM), first in-first out (FIFO) method, and a new RSP-based mutation strategy. The analysis of strategy effectiveness demonstrates that the strategies for enhancing jSO proposed in the work significantly enhance the performance of jSO. Statistical analysis of the results verifies that APSM-jSO outperforms five state-of-the-art DE-based algorithms (ELSHADE-SPACMA, EB-LSHADE, LSHADE-RSP, mL-SHADE, and MadDE), is superior to four top winners of the IEEE CEC competitions (AGSK, APGSK-IMODE, HSES, and IMODE), and is not inferior to the top method of the IEEE CEC 2017 competition (EBOwithCMAR). In summary, APSM-jSO proposed in this study is a promising jSO variant that effectively further enhances the overall performance of jSO, even if jSO is a very competitive algorithm.

In future work, the parameters of APSM-jSO can be further tuned to fully develop its performance. Moreover, APSM-jSO will be utilized to solve UAV swarm operations real-world problems.

Author agreement

The authors claim that none of the material in the paper has been published or is under consideration for publication elsewhere, and no conflict of interest exits in the submission of this manuscript, and the manuscript is approved by all authors for publication.

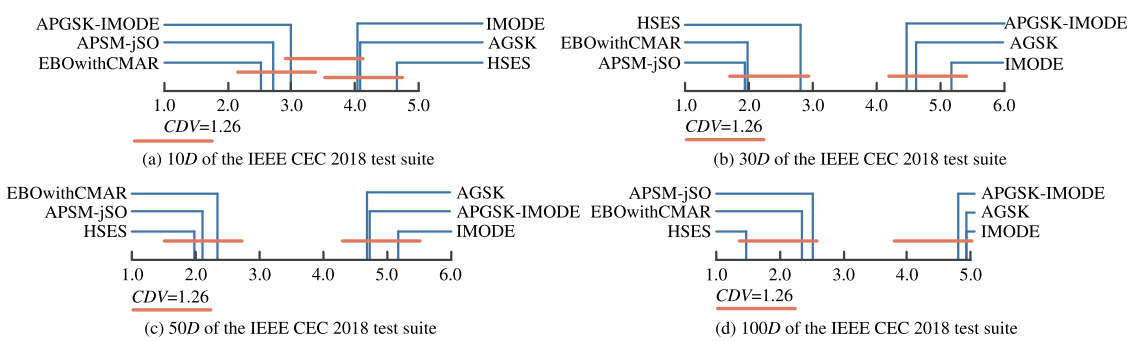


Fig. 17. Multiple comparisons using *CDV* to evaluate APSM-jSO and five winners of IEEE CEC competitions.

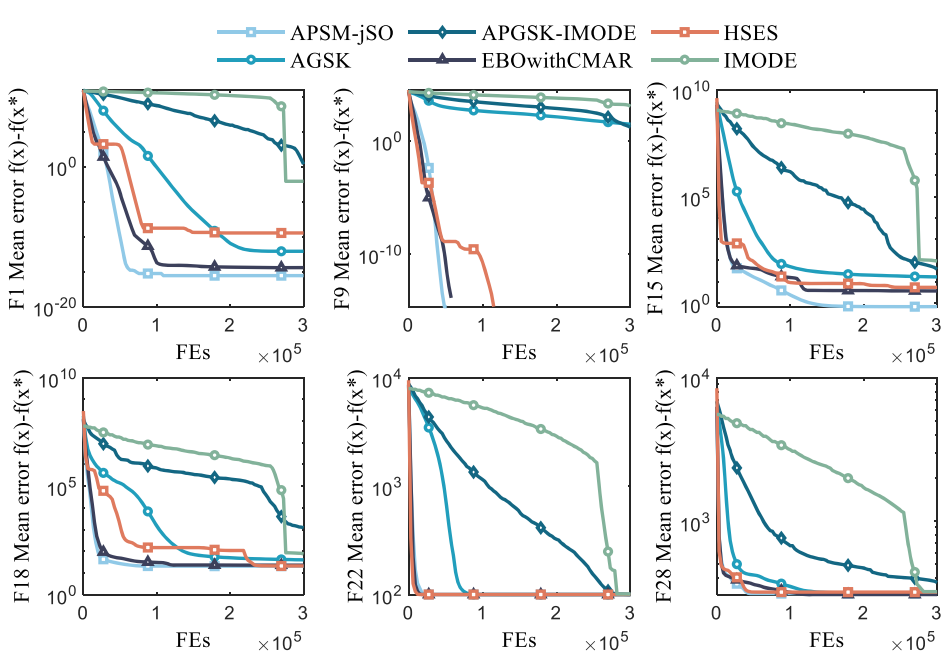


Fig. 18. Convergence graph of APSM-jSO and five winners of the IEEE CEC competitions ($D = 30$).

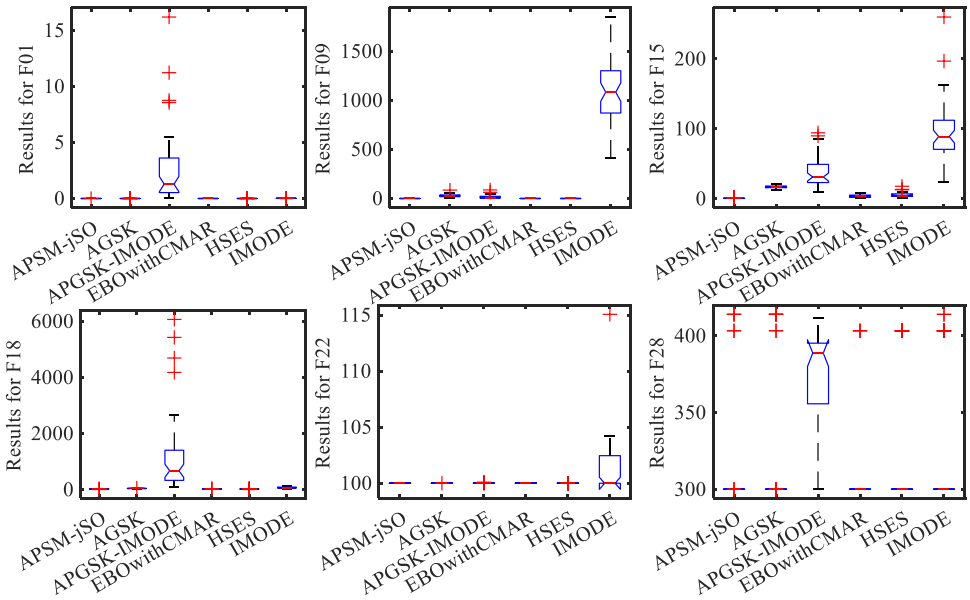


Fig. 19. Box graph of results derived from APSM-jSO and five winners of the IEEE CEC competitions ($D = 30$).

Table 11
Computational complexity of APSM-jSO, AGSK, APGSK-IMODE, EBOwithCMAR, HSES, and IMODE.

Algorithms	$\bar{T}_2$/s				$(\bar{T}_2 - T_1)/T_0$			
	10D	30D	50D	100D	10D	30D	50D	100D
APSM-jSO	0.3456	0.6841	1.1559	3.0301	25.4856	**36.9170**	**50.1415**	**83.2115**
AGSK	0.6254	1.1527	1.7829	4.1452	51.6539	80.7302	108.7722	187.4768
APGSK-IMODE	0.2529	1.8090	2.4217	4.5171	**16.8205**	142.0958	168.5008	222.2549
EBOwithCMAR	1.8741	2.5367	3.4440	7.0439	168.4080	210.1331	264.0821	458.5121
HSES	0.4976	0.9439	2.0039	6.0879	39.7003	61.2071	129.4308	369.1204
IMODE	0.8805	1.1973	1.8236	4.5917	75.5012	84.9039	112.5718	229.2278

T_0=0.0107 s, T_1=0.0730 s (10D), T_1=0.2893s (30D), T_1=0.6196 s (50D). T_1=2.1401s (100D).

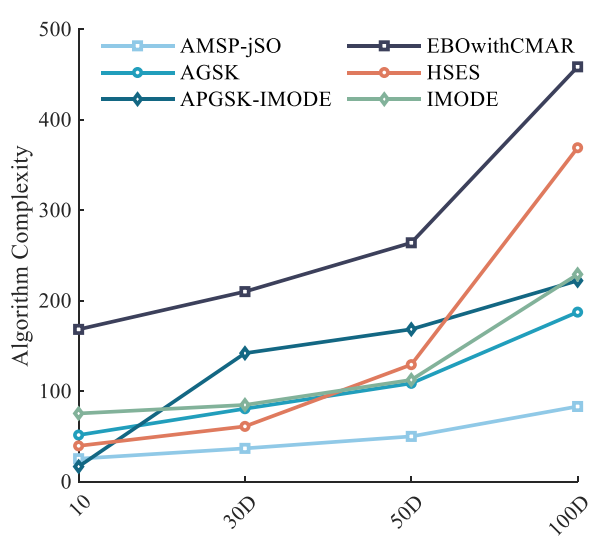


Fig. 20. Computational Complexity of APSM-jSO and five winners of the IEEE CEC competitions.

CRediT authorship contribution statement

Yintong Li: Conceptualization, Methodology, Data curation, Software, Writing – original draft, Writing – review & editing. **Tong Han:** Conceptualization, Investigation, Formal analysis, Funding acquisition, Writing – review & editing. **Huan Zhou:** Conceptualization, Software, Validation, Funding acquisition, Writing – review & editing. **Yujie Wei:** Software, Formal analysis, Validation, Visualization. **Yuan Wang:** Investigation, Funding acquisition, Validation, Writing – review & editing. **Mulai Tan:** Formal analysis, Investigation, Visualization. **Changqiang Huang:** Supervision, Project administration, Writing – review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgement

This work was supported in part by the National Natural Science Foundation of China [grant number 62101590] and the Natural Science Foundation of Shaanxi Province [grant numbers 2021JM-224 and 2022JQ-584].

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.swevo.2023.101283.

References

[1] G.G. Wang, Moth search algorithm: a bio-inspired metaheuristic algorithm for global optimization problems, Memetic. Comput 10 (2018) 151–164, https://doi.org/10.1007/s12293-016-0212-3.

[2] M. Liu, W. Huang, Differential evolution enhanced with multiple dimensional scaling, in: ICEIEC 2019 - Proceedings of 2019 IEEE 9th International Conference on Electronics Information and Emergency Communication, IEEE, 2019, pp. 461–464, https://doi.org/10.1109/ICEIEC.2019.8784579.

[3] T.J. Choi, C.W. Ahn, An improved LSHADE-RSP algorithm with the Cauchy perturbation: iLSHADE-RSP, Knowl. Based Syst 215 (2021), 106628, https://doi.org/10.1016/j.knosys.2020.106628.

[4] Y. Lu, U. Benlic, Q. Wu, A highly effective hybrid evolutionary algorithm for the covering salesman problem, Inf. Sci. (Ny) 564 (2021) 144–162, https://doi.org/10.1016/j.ins.2021.02.053.

[5] K.M. Sallam, M.A. Hossain, R.K. Chakrabortty, M.J. Ryan, An improved gaining-sharing knowledge algorithm for parameter extraction of photovoltaic models, Energy Convers. Manage. 237 (2021), 114030, https://doi.org/10.1016/j.enconman.2021.114030.

[6] Y. Liang, X. Wang, H. Zhao, T. Han, Z. Wei, Y. Li, A covariance matrix adaptation evolution strategy variant and its engineering application, App. Soft Comput. J. 83 (2019), 105680, https://doi.org/10.1016/j.asoc.2019.105680.

[7] P.P. Biswas, P.N. Suganthan, G.A.J. Amaratunga, Optimal placement of wind turbines in a windfarm using L-SHADE algorithm, in: 2017 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2017, pp. 83–88, https://doi.org/10.1109/CEC.2017.7969299.

[8] M. Hamdi, L. Idomghar, M. Chaoui, A. Kachouri, An improved adaptive differential evolution optimizer for non-convex economic dispatch problems, Appl. Soft. Comput 85 (2019), 105868, https://doi.org/10.1016/j.asoc.2019.105868.

[9] R. Storn, K. Price, Differential evolution - a simple and efficient heuristic for global optimization over continuous spaces, J. Global Optim. 11 (1997) 341–359, https://doi.org/10.1023/A:1008202821328.

[10] M.Pant Bilal, H. Zaheer, L. Garcia-Hernandez, A. Abraham, Differential evolution: a review of more than two decades of research, Eng. Appl. Artif. Intell 90 (2020), 103479, https://doi.org/10.1016/j.engappai.2020.103479.

[11] Y. Li, T. Han, S. Tang, C. Huang, H. Zhou, Y. Wang, An improved differential evolution by hybridizing with estimation-of-distribution algorithm, Inf. Sci. (Ny) 619 (2023) 439–456, https://doi.org/10.1016/j.ins.2022.11.029.

[12] J. Brest, S. Greiner, B. Bošković, M. Mernik, V. Zumer, Self-adapting control parameters in differential evolution: a comparative study on numerical benchmark problems, IEEE Trans. Evol. Comput. 10 (2006) 646–657, https://doi.org/10.1109/TEVC.2006.872133.

[13] R. Tanabe, A. Fukunaga, Evaluating the performance of SHADE on CEC 2013 benchmark problems, in: 2013 IEEE Congress on Evolutionary Computation 2013, CEC, 2013, pp. 1952–1959, https://doi.org/10.1109/CEC.2013.6557798.

[14] J. Zhang, A.C. Sanderson, JADE: adaptive differential evolution with optional external archive, IEEE Transac. Evolution Comput 13 (2009) 945–958, https://doi.org/10.1109/TEVC.2009.2014613.

[15] R. Tanabe, A.S. Fukunaga, Improving the search performance of SHADE using linear population size reduction, in: 2014 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2014, pp. 1658–1665, https://doi.org/10.1109/CEC.2014.6900380.

[16] J. Brest, M.S. Maučec, B. Bošković, IL-SHADE: improved L-SHADE algorithm for single objective real-parameter optimization, in: 2016 IEEE Congress on Evolutionary Computation 2016, CEC, 2016, pp. 1188–1195, https://doi.org/10.1109/CEC.2016.7743922.

[17] J. Brest, M.S. Maucec, B. Boskovic, Single objective real-parameter optimization: algorithm jSO, in: 2017 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2017, pp. 1311–1318, https://doi.org/10.1109/CEC.2017.7969456.

[18] N.H. Awad, M.Z. Ali, P.N. Suganthan, R.G. Reynolds, An ensemble sinusoidal parameter adaptation incorporated with L-SHADE for solving CEC2014 benchmark problems, in: 2016 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2016, pp. 2958–2965, https://doi.org/10.1109/CEC.2016.7744163.

[19] N.H. Awad, M.Z. Ali, P.N. Suganthan, Ensemble sinusoidal differential covariance matrix adaptation with Euclidean neighborhood for solving CEC2017 benchmark problems, in: 2017 IEEE Congress on Evolutionary Computation, CEC 2017 - Proceedings, 2017, pp. 372–379, https://doi.org/10.1109/CEC.2017.7969336.

[20] V. Stanovov, S. Akhmedova, E. Semenkin, LSHADE algorithm with rank-based selective pressure strategy for solving CEC 2017 benchmark problems, in: 2018 IEEE Congress on Evolutionary Computation, CEC 2018 - Proceedings, IEEE, 2018, pp. 1–8, https://doi.org/10.1109/CEC.2018.8477977.

[21] Y. Wang, Z. Cai, Q. Zhang, Differential evolution with composite trial vector generation strategies and control parameters, IEEE Transact. Evolutiona. Computat. 15 (2011) 55–66, https://doi.org/10.1109/TEVC.2010.2087271.

[22] F. Zhao, L. Zhao, L. Wang, H. Song, A collaborative LSHADE algorithm with comprehensive learning mechanism, Appl. Soft Comput 96 (2020), 106609, https://doi.org/10.1016/j.asoc.2020.106609.

[23] A.W. Mohamed, A.A. Hadi, A.K. Mohamed, Differential evolution mutations: taxonomy, comparison and convergence analysis, IEEE Access 9 (2021) 68629–68662, https://doi.org/10.1109/ACCESS.2021.3077242.

[24] L. Cui, G. Li, Z. Zhu, Q. Lin, K.C. Wong, J. Chen, N. Lu, J. Lu, Adaptive multiple-elites-guided composite differential evolution algorithm with a shift mechanism, Inf. Sci. (Ny) 422 (2018) 122–143, https://doi.org/10.1016/j.ins.2017.09.002.

[25] A.W. Mohamed, P.N. Suganthan, Real-parameter unconstrained optimization based on enhanced fitness-adaptive differential evolution algorithm with novel mutation, Soft. Comput 22 (2018) 3215–3235, https://doi.org/10.1007/s00500-017-2777-2.

[26] A.W. Mohamed, A.K. Mohamed, Adaptive guided differential evolution algorithm with novel mutation for numerical optimization, Internati. J. Machine Learn. Cyberneti. 10 (2019) 253–277, https://doi.org/10.1007/s13042-017-0711-7.

[27] G. Wu, X. Shen, H. Li, H. Chen, A. Lin, P.N. Suganthan, Ensemble of differential evolution variants, Inf. Sci. (Ny) 423 (2018) 172–186, https://doi.org/10.1016/j.ins.2017.09.053.

[28] R. Mallipeddi, P.N. Suganthan, Q.K. Pan, M.F. Tasgetiren, Differential evolution algorithm with ensemble of parameters and mutation strategies, App. Soft Comput. J. 11 (2011) 1679–1696, https://doi.org/10.1016/j.asoc.2010.04.024.

[29] J. Cheng, Z. Pan, H. Liang, Z. Gao, J. Gao, Differential evolution algorithm with fitness and diversity ranking-based mutation operator, Swarm Evol. Comput 61 (2021), 100816, https://doi.org/10.1016/j.swevo.2020.100816.

[30] W. Yi, Y. Chen, Z. Pei, J. Lu, Adaptive differential evolution with ensembling operators for continuous optimization problems, Swarm Evol. Comput 69 (2022), 100994, https://doi.org/10.1016/j.swevo.2021.100994.

[31] A. Viktorin, R. Senkerik, M. Pluhacek, T. Kadavy, A. Zamuda, Distance based parameter adaptation for success-history based differential evolution, Swarm Evol. Comput 50 (2019), 100462, https://doi.org/10.1016/j.swevo.2018.10.013.

[32] A.W. Mohamed, A.A. Hadi, A.M. Fattouh, K.M. Jambi, LSHADE with semi-parameter adaptation hybrid with CMA-ES for solving CEC 2017 benchmark problems, in: 2017 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2017, pp. 145–152, https://doi.org/10.1109/CEC.2017.7969307.

[33] M. Liu, X. Yao, Y. Li, Hybrid whale optimization algorithm enhanced with Lévy flight and differential evolution for job shop scheduling problems, Appl. Soft. Comput 87 (2020), 105954, https://doi.org/10.1016/j.asoc.2019.105954.

[34] E.V. Altay, B. Alatas, Differential evolution and sine cosine algorithm based novel hybrid multi-objective approaches for numerical association rule mining, Inf. Sci. (Ny) 554 (2021) 198–221, https://doi.org/10.1016/j.ins.2020.12.055.

[35] J.-.F. Yeh, T.-.Y. Chen, T.-.C. Chiang, Modified L-SHADE for Single Objective Real-Parameter Optimization, in: 2019 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2019, pp. 381–386, https://doi.org/10.1109/CEC.2019.8789991.

[36] A.W. Mohamed, A.A. Hadi, K.M. Jambi, Novel mutation strategy for enhancing SHADE and LSHADE algorithms for global numerical optimization, Swarm Evolut. Comput. 50 (2019) 100455. https://doi.org/10.1016/j.swevo.2018.10.006.

[37] A.A. Hadi, A.W. Mohamed, K.M. Jambi, Single-objective real-parameter optimization: enhanced LSHADE-SPACMA algorithm, Stud. in Computat.. Intellige. (2021) 103–121, https://doi.org/10.1007/978-3-030-58930-1_7.

[38] S. Biswas, D. Saha, S. De, A.D. Cobb, S. Das, B.A. Jalaian, Improving differential evolution through bayesian hyperparameter optimization, in: 2021 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2021, pp. 832–840, https://doi.org/10.1109/CEC45853.2021.9504792.

[39] N.H. Awad, M.Z. Ali, J. Liang, B.Y. Qu, P.N. Suganthan, Problem definitions and evaluation criteria for the CEC 2017 special session and competition on real-parameter optimization, 2016.

[40] J.M. Davenport, Approximations of the critical region of the friedman statistic, Communicat. Statistics 9 (1980) 571–595, https://doi.org/10.1080/03610928008827904.

[41] Y. Li, T. Han, H. Zhou, S. Tang, H. Zhao, A novel adaptive L-SHADE algorithm and its application in UAV swarm resource configuration problem, Inf. Sci. (Ny) 606 (2022) 350–367, https://doi.org/10.1016/j.ins.2022.05.058.

[42] A.W. Mohamed, A.A. Hadi, A.K. Mohamed, N.H. Awad, Evaluating the performance of adaptive gainingsharing knowledge based algorithm on CEC 2020 benchmark problems, in: 2020 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2020, pp. 1–8, https://doi.org/10.1109/CEC48606.2020.9185901.

[43] A.W. Mohamed, A.A. Hadi, P. Agrawal, K.M. Sallam, A.K. Mohamed, Gaining-sharing knowledge based algorithm with adaptive parameters hybrid with IMODE algorithm for solving CEC 2021 benchmark problems, in: 2021 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2021, pp. 841–848, https://doi.org/10.1109/CEC45853.2021.9504814.

[44] A. Kumar, R.K. Misra, D. Singh, Improving the local search capability of effective butterfly optimizer using covariance matrix adapted retreat phase, in: 2017 IEEE Congress on Evolutionary Computation, CEC 2017 - Proceedings, IEEE, 2017, pp. 1835–1842, https://doi.org/10.1109/CEC.2017.7969524.

[45] G. Zhang, Y. Shi, Hybrid sampling evolution strategy for solving single objective bound constrained problems, in: 2018 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2018, pp. 1–7, https://doi.org/10.1109/CEC.2018.8477908.

[46] K.M. Sallam, S.M. Elsayed, R.K. Chakrabortty, M.J. Ryan, Improved multi-operator differential evolution algorithm for solving unconstrained problems, in: 2020 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2020, pp. 1–8, https://doi.org/10.1109/CEC48606.2020.9185577.